

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»

Навчально-науковий інститут комп'ютерних систем

Кафедра інженерії програмного забезпечення

Ілля ВЕРБАНОВ

(група АС-222)

КВАЛІФІКАЦІЙНА РОБОТА БАКАЛАВРА
Вебзастосунок для пошуку загублених домашніх тварин

Спеціальність:

121 Інженерія програмного забезпечення

Освітньо-професійна програма:

Інженерія програмного забезпечення

Керівник:

Надія БЕГЛОВА, асистент каф. ІПЗ

Одеса – 2026

АНОТАЦІЯ

Вербанов І. Г. Вебзастосунок для пошуку загублених домашніх тварин : кваліфікаційна робота бакалавра за спеціальністю «121 Інженерія програмного забезпечення» / Ілля Геннадійович Вербанов ; керівник Надія Миколаївна Беглова – Одеса : Нац. ун-т «Одес. політехніка», 2026. – 70 с.

Кваліфікаційна робота містить основну текстову частину на 59 сторінках, список використаних джерел з 14 найменувань на 2 сторінках, додатки на 2 сторінках.

Метою кваліфікаційної роботи є підвищення ефективності первинного зіставлення оголошень про загублених і знайдених тварин шляхом розроблення вебзастосунку, що автоматично формує ранжований перелік ймовірних збігів за видом тварини, географічною відстанню та часовим вікном події. Ефективність оцінюється часткою випадків, коли істинний збіг потрапляє в перші K позицій переліку ($\text{Hit}@K$, $K = 1, 3, 5$), середнім оберненим рангом істинного збігу (MRR) і часом формування переліку; досягнення мети підтверджується експериментально.

У роботі розроблено вебзастосунок, який дозволяє власникам публікувати оголошення про загублену тварину, а особам, які знайшли тварину, — оголошення про знахідку. Реалізовано алгоритм формування ранжованого переліку оголошень-кандидатів, що оцінює збіг за видом тварини, відстанню між місцями події за формулою гаверсинуса та різницею дат. Запропонований кандидат підтверджується людиною: після підтвердження зв'язку контактні дані сторін взаємно розкриваються, що дозволяє організувати повернення тварини.

Систему реалізовано із застосуванням платформи Node.js, фреймворку NestJS, бібліотеки React, ORM Drizzle та системи керування базами даних PostgreSQL. У процесі розробки проведено аналіз предметної області та аналогів, сформовано функціональні та нефункціональні вимоги, виконано планування проекту, спроектовано архітектуру системи, структуру бази даних і користувацький інтерфейс, проведено функціональне та нефункціональне тестування.

Результатом кваліфікаційної роботи є вебзастосунок для пошуку загублених тварин, який може бути використаний волонтерськими спільнотами та притулками для автоматизації первинного зіставлення оголошень. Проведене тестування підтвердило коректність роботи системи, а експериментальне дослідження на модельному наборі даних — досягнення мети: істинний збіг потрапляє у перші позиції ранжованого переліку, а час формування переліку задовольняє вимогу продуктивності.

Ключові слова: вебзастосунок, пошук загублених тварин, алгоритм зіставлення, формула гаверсинуса, ранжування кандидатів, база даних, NestJS, React.

ABSTRACT

Verbanov I. H. Web application for finding lost pets : Bachelor's qualification work in the specialty «121 Software Engineering» / Illia Hennadiiovych Verbanov ; supervisor Nadiia Mykolaivna Behlova – Odesa : Odesa Polytechnic Nat. Univ., 2026. – 70 pages.

The qualification work contains the main text part on 59 pages, a list of used sources with 14 titles on 2 pages, and appendices on 2 pages.

The aim of the qualification work is to improve the effectiveness of the initial matching of lost- and found-pet reports by developing a web application that automatically builds a ranked list of likely matches by animal species, geographic distance, and the time window of the event. Effectiveness is assessed by the share of cases in which the true match appears among the top K positions of the list (Hit@K, K = 1, 3, 5), by the mean reciprocal rank of the true match (MRR), and by the time to build the list; the achievement of the aim is confirmed experimentally.

The work presents a web application that allows owners to publish reports about a lost pet and finders to publish reports about a found animal. A matching algorithm is implemented that builds a ranked list of candidate reports, evaluating the match by species equality, the distance between event locations computed with the haversine formula, and the difference of dates. A proposed candidate is confirmed by a human: after the link is confirmed, the contact details of both parties are mutually revealed so that the return of the pet can be arranged.

The system is implemented using the Node.js platform, the NestJS framework, the React library, the Drizzle ORM, and the PostgreSQL database management system. During the development, the subject area and existing solutions were analyzed, functional and non-functional requirements were defined, project planning was carried out, the system architecture, database structure, and user interface were designed, and functional and non-functional testing was performed.

The result of the qualification work is a web application for finding lost pets that can be used by volunteer communities and shelters to automate the initial matching of reports. Testing confirmed the correctness of the system,

and an experiment on a model dataset confirmed that the aim was achieved: the true match appears among the top positions of the ranked list, and the list is built within the performance requirement.

Keywords: web application, lost pet search, matching algorithm, haversine formula, candidate ranking, database, NestJS, React.

ЗМІСТ

Вступ	8
1 Специфікація вимог до вебзастосунку	11
1.1 Опис предметної області	11
1.2 Аналіз аналогів	12
1.3 Функціональні вимоги	16
1.4 Нефункціональні вимоги	17
1.5 Вимоги до інтерфейсу користувача	18
1.6 Варіанти використання системи	18
Висновки до розділу 1	24
2 Планування проєкту з розробки вебзастосунку	25
2.1 Визначення масштабу проєкту	25
2.2 Оцінка розміру проєкту	26
2.3 Календарний план робіт	29
2.4 Оцінка ризиків	30
Висновки до розділу 2	31
3 Проєктування вебзастосунку	32
3.1 Проєктування моделі оголошення та зв'язку	32
3.2 Архітектура вебзастосунку	33
3.3 Проєктування структури бази даних	35
3.4 Проєктування на рівні класів	38
Висновки до розділу 3	40
4 Програмна реалізація вебзастосунку	41
4.1 Алгоритм зіставлення оголошень	41
4.2 Ранжування кандидатів	43
4.3 Вибір інструментів розробки	45
4.4 Реалізація сервісу зіставлення оголошень	46
Висновки до розділу 4	47
5 Тестування вебзастосунку	49
5.1 Процес тестування	49
5.2 Функціональне тестування	50
5.3 Тестування нефункціональних вимог	52

Висновки до розділу 5	53
6 Експериментальне дослідження ефективності зіставлення	54
6.1 Методика експерименту та модельний набір даних	54
6.2 Оцінювання метрик ранжування	55
6.3 Порівняння з базовою лінією та час формування переліку	56
Висновки до розділу 6	57
7 Експлуатація застосунку	58
7.1 Робота користувача з системою	58
7.2 Приклад зіставлення оголошень про загублену тварину	60
7.3 Розгортання застосунку адміністратором (розробником)	62
Висновки до розділу 7	64
Загальні висновки	65
Список використаних джерел	67
Додаток А Лістинги програмного коду	69

ВСТУП

Актуальність теми роботи. Втрата домашньої тварини є поширеною ситуацією, у якій вирішальне значення має час: чим довше тварина залишається ненайденою, тим менша ймовірність її повернення. За даними міжнародних досліджень (ASPCA, American Humane Association), протягом життя власника втрачає близько 14–15 % собак і близько 15 % котів; із них своїми силами або через сусідів повертаються близько 60 % собак, тоді як для котів цей показник не перевищує 30 %. У межах першого тижня після зникнення додому повертається лише близько третини тварин, після чого ймовірність повернення стрімко знижується. В умовах України ситуація додатково ускладнюється воєнним станом і вимушеною міграцією населення: за оцінками громадських організацій, що опікуються тваринами (UAnimals, «Cіріус», «Happy Paw»), кількість загублених і покинутих домашніх тварин у великих містах зросла у 2–3 рази порівняно з довоєнним періодом, а основними каналами пошуку залишаються неструктуровані дописи у Telegram-каналах, Facebook-групах і на класифайдах на кшталт OLX. Власник, який втратив тварину, і людина, яка її знайшла, зазвичай діють розрізнено: оголошення розміщуються в різних соціальних мережах, тематичних групах і дошках оголошень, не мають єдиної структури та не зіставляються між собою автоматично. Зіставлення оголошення про втрату з оголошенням про знахідку виконується людьми вручну, спирається на випадковий перегляд стрічки і значною мірою залежить від того, чи побачать обидві сторони допис одна одної. Це збільшує час пошуку і знижує шанси на возз'єднання тварини з власником.

Практична цінність розробленого вебзастосунку полягає в тому, що він замінює неструктурований ручний пошук на автоматичне геопросторове та видо-орієнтоване зіставлення оголошень: одразу після публікації оголошення про знахідку власник загубленої тварини у тій самій географічній зоні отримує ранжований перелік ймовірних збігів без необхідності переглядати десятки сторонніх дописів. Цільовою аудиторією є власники домашніх тварин, волонтери зоозахисних організацій та небайдужі громадяни, які знайшли тварину. Очікуваним практичним ефек-

том є скорочення середнього часу від зникнення до возз'єднання та підвищення частки повернутих тварин, передусім за рахунок раннього виявлення збігу у перші 24–72 години — найкритичніший проміжок часу для пошуку.

Актуальність роботи зумовлена такими факторами:

- поширеністю випадків втрати домашніх тварин і високою соціальною значущістю їх повернення;
- розрізненістю наявних каналів публікації оголошень і відсутністю єдиної структурованої моделі оголошення;
- необхідністю скорочення часу пошуку за рахунок автоматичного зіставлення оголошень про втрату та про знахідку;
- потребою у захисті персональних контактних даних до моменту підтвердження збігу;
- розвитком вебтехнологій, які дозволяють реалізувати геопросторове зіставлення без спеціалізованих картографічних розширень бази даних.

Темою кваліфікаційної роботи є розробка вебзастосунку, що забезпечує публікацію структурованих оголошень про загублених і знайдених домашніх тварин та автоматичне формування переліку ймовірних збігів між ними.

Мета і задачі роботи. Метою роботи є підвищення ефективності первинного зіставлення оголошень про загублених і знайдених домашніх тварин шляхом розроблення вебзастосунку, що автоматично формує ранжований перелік ймовірних збігів. Ефективність зіставлення оцінюється часткою випадків, коли істинний збіг потрапляє в перші K позицій переліку ($\text{Hit}@K$, $K = 1, 3, 5$), середнім оберненим рангом істинного збігу (MRR) і часом формування переліку; досягнення мети підтверджується експериментально. Для досягнення поставленої мети необхідно вирішити такі задачі:

- дослідити предметну область пошуку загублених домашніх тварин;
- провести аналіз існуючих рішень і підходів;
- сформулювати функціональні та нефункціональні вимоги до системи;

- виконати планування проєкту та оцінювання ризиків;
- розробити алгоритм зіставлення та ранжування оголошень-кандидатів;
- спроектувати архітектуру застосунку, структуру бази даних і користувацький інтерфейс;
- реалізувати вебзастосунок із використанням сучасних технологій;
- провести тестування функціональних і нефункціональних вимог системи;
- провести експериментальне дослідження ефективності розробленого зіставлення на модельному наборі даних.

Об'єкт роботи. Об'єктом роботи є процес розроблення програмного забезпечення для структурованого подання та автоматичного зіставлення оголошень про загублених і знайдених домашніх тварин.

Предмет роботи. Предметом роботи є моделі даних, метод і алгоритм зіставлення та ранжування оголошень, а також програмні засоби (архітектура, компоненти й інтерфейси вебзастосунку), що реалізують автоматичне формування ранжованого переліку ймовірних збігів із подальшим підтвердженням зв'язку людиною.

1 СПЕЦИФІКАЦІЯ ВИМОГ ДО ВЕБЗАСТОСУНКУ

1.1 Опис предметної області

Втрата домашньої тварини зачіпає значну частину власників протягом життя тварини. Успіх повернення залежить від того, наскільки швидко інформація про втрату досягне людини, яка тварину знайшла, і навпаки. Дослідження практик повернення тварин показують, що більшість успішних випадків припадає на перші дні після зникнення та на невелику відстань від місця, де тварину востаннє бачили [1].

Сьогодні власник, який втратив тварину, найчастіше діє так: розміщує допис у місцевих групах соціальних мереж, на дошках оголошень, звертається до притулків і ветеринарних клінік. Людина, яка знайшла тварину, діє симетрично, але незалежно. Оголошення обох сторін потрапляють у різні канали, мають довільну структуру і не зіставляються між собою. Збіг виявляється випадково — якщо обидві сторони переглянуть допис одна одної у стрічці.

Така організація процесу має кілька недоліків. По-перше, оголошення не структуровані: вид тварини, місце та дата події часто наведені у вільній формі, що ускладнює пошук. По-друге, зіставлення виконується вручну і залежить від обсягу стрічки та уважності читача. По-третє, контактні дані публікуються відкрито, що створює ризик зловживань ще до того, як збіг підтверджено.

У наукових джерелах і практичних посібниках наголошується, що ключовими ознаками для зіставлення є вид тварини, географічна близькість місць події та невеликий проміжок часу між втратою і знахідкою [1, 2]. Саме ці ознаки піддаються формалізації та автоматичному порівнянню.

Таким чином, процес пошуку загублених тварин потребує єдиної структурованої моделі оголошення та автоматичного зіставлення оголошень про втрату і про знахідку з подальшим підтвердженням збігу людиною.

1.2 Аналіз аналогів

Аналіз наявних рішень дозволяє виявити вдалі підходи до побудови систем пошуку загублених тварин, а також їхні недоліки, які слід урахувати під час розробки. Особливу увагу приділено таким аспектам:

- структурованості оголошення про тварину;
- наявності автоматичного зіставлення оголошень про втрату та про знахідку;
- урахуванню географічної відстані та дати події;
- захисту контактних даних до підтвердження збігу;
- зручності користувацького інтерфейсу.

Для порівняння обрано такі аналоги:

- сервіс сповіщень про втрачених тварин PawBoost [3];
- некомерційна база загублених і знайдених тварин Pet FBI [4];
- тематичні спільноти в соціальних мережах і розділи дощок оголошень [5].

PawBoost — сервіс, що дозволяє опублікувати оголошення про втрачену або знайдену тварину та платно поширити його у вигляді сповіщень серед користувачів поблизу (рис. 1.1). Сервіс має структуровану форму оголошення та велику аудиторію.

Рисунок 1.1 – Інтерфейс публікації оголошення в сервісі PawBoost

Переваги:

- структурована форма оголошення;
- поширення сповіщень за геолокацією;
- велика база користувачів.

Недоліки:

- зіставлення втрати та знахідки не автоматизоване;
- розширене поширення є платним;
- контактні дані відкриваються без підтвердження збігу.

Pet FBI — некомерційна база загублених і знайдених тварин із безкоштовною публікацією оголошень і пошуком за фільтрами (рис. 1.2).

The screenshot displays the Pet FBI website. At the top, there is a navigation bar with links for 'SEARCH REPORTS', 'POST A REPORT', and 'MORE INFO', along with a 'EDIT MY REPORTS' button. The main heading reads 'Search Our Database of Lost and Found Pet Reports'. Below this, there are two columns: 'Lost or Found a Pet? Enter a Report' with a 'START A REPORT' button, and 'We Will Send You Potential Matches' with a 'GO TO DASHBOARD' button. A central section titled 'Search the Pet FBI Database by Location and Species' includes a search filter section on the left and a 'Search Result Map' on the right. The search filters include a location toggle (YES/NO), a search input field, and a species dropdown menu currently set to 'Dog'. The map shows a geographical view of the search area.

Рисунок 1.2 – Результати пошуку в базі Pet FBI

Переваги:

- безкоштовна публікація оголошень;
- пошук за видом і місцем;
- некомерційна спрямованість.

Недоліки:

- зіставлення виконується користувачем вручну;
- відсутнє ранжування ймовірних збігів;
- немає механізму підтвердження зв'язку із приховуванням контактів.

Тематичні спільноти в соціальних мережах і розділи дощок оголошень використовуються найчастіше через нульовий поріг входу (рис. 1.3).

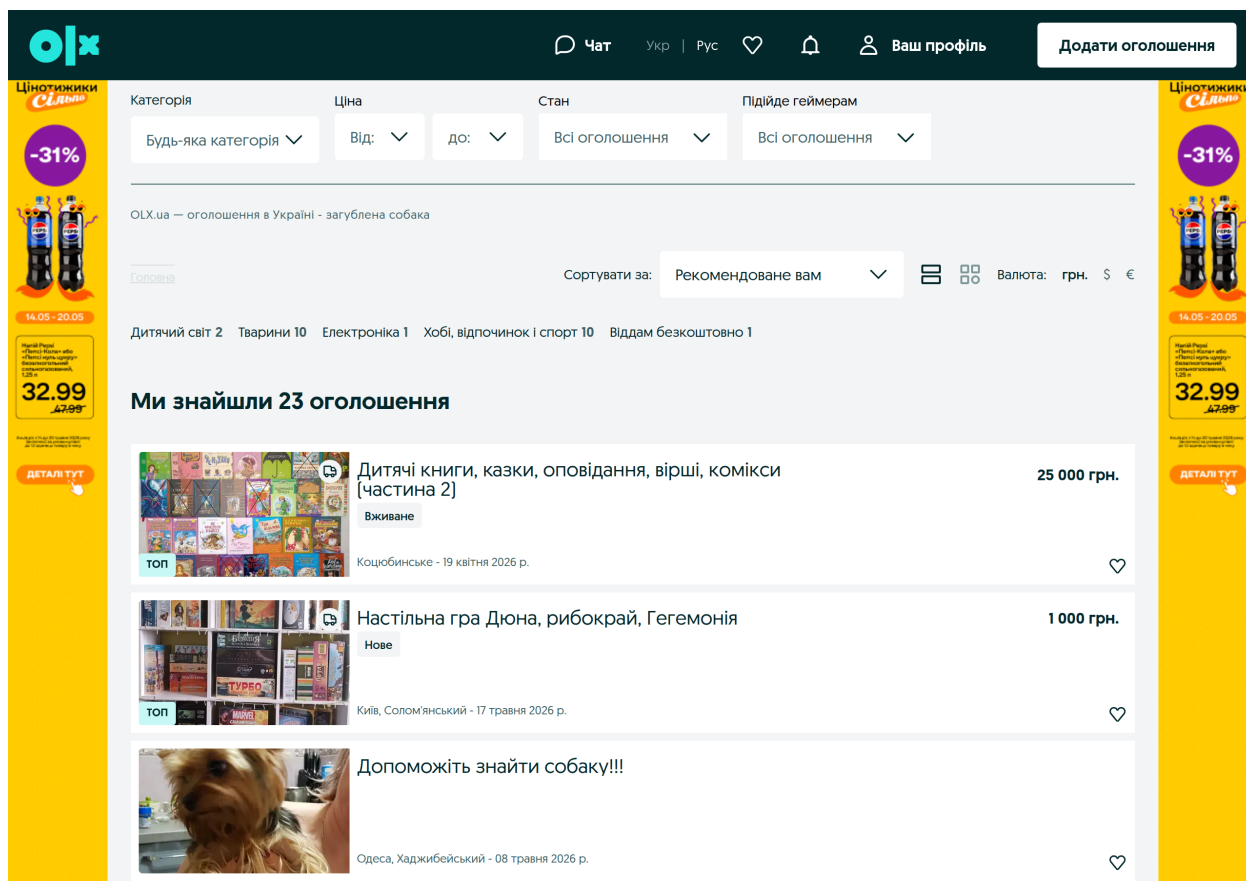


Рисунок 1.3 – Допис про загублену тварину в тематичній спільноті

Переваги:

- нульовий поріг входу;
- локальна аудиторія;
- швидка публікація.

Недоліки:

- довільна структура оголошення;
- відсутнє автоматичне зіставлення;
- контактні дані публікуються відкрито.

Для наочного представлення результатів виконано порівняння аналогів за основними критеріями. Результати порівняння наведено в таблиці 1.1.

Таблиця 1.1 – Порівняння аналогів

Критерій	PawBoost	Pet FBI	Спільноти
Структурована модель оголошення	+	+	–
Автоматичне зіставлення втрати та знахідки	–	–	–
Урахування відстані та дати	частково	частково	–
Ранжування кандидатів	–	–	–
Підтвердження збігу людиною	–	–	–
Приховування контактів до підтвердження	–	–	–
Безкоштовність базових функцій	частково	+	+

Аналіз показує, що наявні рішення забезпечують публікацію оголошень, але мають спільні прогалини: жодне з них не виконує автоматичного зіставлення оголошень про втрату та про знахідку, не ранжує ймовірні збіги і не приховує контактні дані до підтвердження. Саме ці прогалини визначають вимоги до розроблюваної системи: структурована модель оголошення; автоматичне зіставлення з урахуванням виду тварини, географічної відстані та дати події; ранжування кандидатів; підтвердження збігу людиною; приховування контактів до підтвердження. Сформульовані на цій основі вимоги конкретизовано в підрозділах 1.3–1.6.

1.3 Функціональні вимоги

Функціональні вимоги до системи визначено у формі варіантів використання: кожна функціональна вимога відповідає окремому варіанту використання, що описує завершену взаємодію користувача із системою. Основним актором є зареєстрований користувач, який виступає у відносній ролі власника (автор оголошення про втрату) або особи, яка знайшла тварину (автор оголошення про знахідку); окремої ролі адміністратора в межах роботи не передбачено.

Систему визначають вісім варіантів використання:

- реєстрація та вхід користувача;
- публікація оголошення про загублену тварину;
- публікація оголошення про знайдену тварину;
- перегляд і пошук публічної стрічки оголошень із фільтрацією за видом, станом, місцем і періодом;

- перегляд ранжованого переліку оголошень-кандидатів для власного оголошення;
- пропонування зв'язку між оголошенням про втрату та оголошенням про знахідку;
- підтвердження або відхилення запропонованого зв'язку та взаємне розкриття контактних даних;
- зміна стану оголошення (повернуто, вирішено, закрито).

Діаграму варіантів використання та деталізовані сценарії, що описують кожен із них, наведено в підрозділі 1.6.

1.4 Нефункціональні вимоги

Нефункціональні вимоги визначають характеристики якості системи та умови її функціонування. До основних нефункціональних вимог належать:

- продуктивність: формування переліку кандидатів для одного оголошення має виконуватися не більше ніж за 2 секунди за обсягу до десятків тисяч активних оголошень;
- зручність використання: інтерфейс має бути зрозумілим, придатним для роботи з мобільного пристрою та мінімізувати кількість дій для публікації оголошення;
- безпека: паролі зберігаються лише у вигляді геш-значень за алгоритмом argon2id; доступ до даних за замовчуванням заборонено і надається лише автентифікованому користувачеві; контактні дані не потрапляють у публічні подання;
- надійність: некоректні вхідні дані відхиляються на межі системи з повідомленням про помилку; неприпустимі переходи стану оголошення блокуються;
- масштабованість: архітектура має допускати заміну сховища файлів і додавання нових ознак зіставлення без суттєвої переробки наявного коду;
- підтримуваність: програмний код структурований за модулями з визначеними межами та публічним інтерфейсом;

- цілісність даних: пара «оголошення про втрату — оголошення про знахідку» не може мати більше одного зв'язку.

1.5 Вимоги до інтерфейсу користувача

Інтерфейс вебзастосунку має бути зрозумілим, послідовним і придатним для роботи з мобільного пристрою, оскільки оголошення про знахідку часто публікують безпосередньо на місці події. Для цього в системі передбачено такі елементи інтерфейсу:

- публічна стрічка оголошень із фільтрами за видом тварини, станом, місцем і періодом події;
- форма публікації оголошення з послідовними секціями і необов'язковим кроком долучення фотографії;
- сторінка оголошення з двома поданнями: публічним (без контактів) та поданням власника;
- сторінка кандидатів, що відображає для кожного запропонованого оголошення відстань у кілометрах, збіг за видом і різницю дат;
- сторінка зв'язків користувача з можливістю підтвердити або відхилити запропонований зв'язок;
- повідомлення про помилки введення зрозумілою мовою;
- єдина система візуальних токенів (кольори, відступи, типографіка) зі світлою і темною темами;
- подання стану оголошення одночасно піктограмою, підписом і кольором, а не лише кольором, для доступності.

1.6 Варіанти використання системи

Сукупність варіантів використання, визначених у підрозділі 1.3, та їх зв'язок з основним актором подано на діаграмі варіантів використання (рис. 1.4).

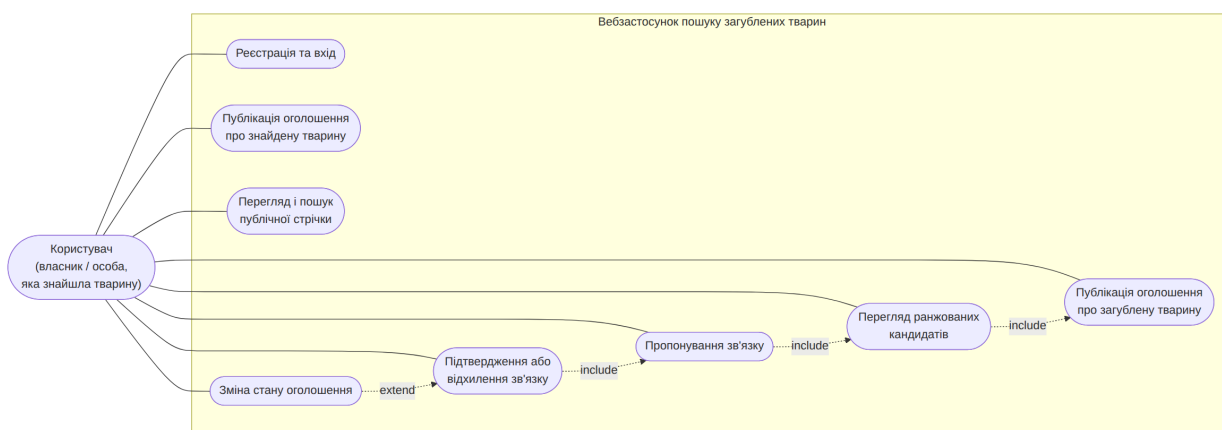


Рисунок 1.4 – Діаграма варіантів використання системи

Кожен варіант використання деталізовано нижче у вигляді сценарію за єдиною структурою. Робота користувача із системою має послідовний характер: реєстрація, публікація оголошення, перегляд ранжованих кандидатів, пропонування зв'язку, підтвердження зв'язку другою стороною та переведення оголошення у завершальний стан.

Сценарій 1.1 – Реєстрація та вхід користувача

Мета: отримати автентифікований доступ до системи. **Основна діюча особа:** користувач. **Область дії:** вебзастосунок пошуку загублених тварин. **Передумови:** користувач не автентифікований і має дійсну електронну пошту. **Тригер:** користувач обирає функцію «Зареєструватися» або «Увійти». **Основний успішний сценарій:**

1. Користувач вводить електронну пошту і пароль.
2. Система перевіряє коректність формату та унікальність пошти.
3. Система створює обліковий запис, зберігаючи лише геш пароля за алгоритмом argon2id.
4. Система видає токен автентифікації.
5. Користувач отримує доступ до публікації та керування власними оголошеннями.

Розширення (альтернативні сценарії): 2а. Якщо пошту вже зайнято, система відмовляє з повідомленням «пошту зайнято» і не створює запис. 2б. Якщо пошта або пароль некоректні за форматом, система відхиляє введення з повідомленням про помилку. 1а. Якщо обліковий запис уже існує, користувач натомість входить: система звіряє геш пароля,

а за невірних даних відмовляє з повідомленням «невірні дані». **Результати (постумови):** створено обліковий запис або відкрито сесію наявного; користувач автентифікований і має токен доступу.

Сценарій 1.2 – Публікація оголошення про загублену тварину

Мета: опублікувати структуроване оголошення про загублену тварину. **Основна діюча особа:** користувач (власник). **Область дії:** вебзастосунок пошуку загублених тварин. **Передумови:** користувач автентифікований. **Тригер:** користувач обирає функцію «Повідомити про тварину». **Основний успішний сценарій:**

1. Користувач обирає тип оголошення «Загублено».
2. Система відображає форму оголошення.
3. Користувач вводить вид тварини, опис, місце та дату події.
4. Користувач за бажанням долучає фотографію.
5. Користувач вводить контактні дані.
6. Користувач підтверджує публікацію.
7. Система перевіряє коректність введених даних.
8. Система зберігає оголошення зі станом «активне».

Розширення (альтернативні сценарії): 7а. Якщо обов'язкові поля не заповнені або дані некоректні, система виводить повідомлення про помилку і не зберігає оголошення. 4а. Якщо завантаження фотографії не вдалося, оголошення зберігається без фотографії, а користувача повідомляють про це. **Результати (постумови):** оголошення про втрату збережено зі станом «активне» і доступне в публічній стрічці без контактних даних.

Сценарій 1.3 – Публікація оголошення про знайдену тварину

Мета: опублікувати структуроване оголошення про знайдену тварину. **Основна діюча особа:** користувач (особа, яка знайшла тварину). **Область дії:** вебзастосунок пошуку загублених тварин. **Передумови:** користувач автентифікований. **Тригер:** користувач обирає функцію «Повідомити про тварину». **Основний успішний сценарій:**

1. Користувач обирає тип оголошення «Знайдено».
2. Система відображає форму оголошення.

3. Користувач вводить вид тварини, опис, місце та дату події.
4. Користувач за бажанням долучає фотографію.
5. Користувач вводить контактні дані.
6. Користувач підтверджує публікацію.
7. Система перевіряє коректність введених даних.
8. Система зберігає оголошення зі станом «активне».

Розширення (альтернативні сценарії): 7а. Якщо обов'язкові поля не заповнені або дані некоректні, система виводить повідомлення про помилку і не зберігає оголошення. 4а. Якщо завантаження фотографії не вдалося, оголошення зберігається без фотографії, а користувача повідомляють про це. **Результати (постумови):** оголошення про знахідку збережено зі станом «активне» і доступне в публічній стрічці без контактних даних.

Сценарій 1.4 – Перегляд і пошук публічної стрічки оголошень

Мета: знайти релевантні оголошення в публічній стрічці. **Основна діюча особа:** користувач (зокрема неавтентифікований відвідувач). **Область дії:** вебзастосунок пошуку загублених тварин. **Передумови:** відсутні, оскільки стрічка публічна. **Тригер:** користувач відкриває публічну стрічку оголошень. **Основний успішний сценарій:**

1. Система відображає перелік активних оголошень без контактних даних.
2. Користувач задає фільтри за видом тварини, станом, місцем і періодом події.
3. Система повертає відфільтрований перелік.
4. Користувач відкриває обране оголошення у публічному поданні без контактних даних.

Розширення (альтернативні сценарії): 3а. Якщо за заданими фільтрами оголошень немає, система повідомляє про відсутність результатів. **Результати (постумови):** користувачеві показано відфільтрований перелік активних оголошень без контактних даних; стан даних не змінено.

Сценарій 1.5 – Перегляд кандидатів для оголошення

Мета: отримати перелік ймовірних збігів для власного оголошення. **Основна діюча особа:** користувач (автор оголошення). **Область дії:** вебзастосунок пошуку загублених тварин. **Передумови:** користувач автентифікований і є автором оголошення. **Тригер:** користувач відкриває сторінку кандидатів свого оголошення. **Основний успішний сценарій:**

1. Користувач відкриває власне оголошення.
2. Користувач обирає функцію перегляду кандидатів.
3. Система перевіряє, що користувач є автором оголошення.
4. Система формує перелік оголошень протилежного типу.
5. Система обчислює для кожного оголошення відстань, збіг за видом і різницю дат.
6. Система впорядковує перелік за ступенем відповідності.
7. Система відображає ранжований перелік кандидатів.

Розширення (альтернативні сценарії): За. Якщо користувач не є автором оголошення, система відмовляє у доступі. ба. Якщо відповідних оголошень немає, система повідомляє про відсутність кандидатів. **Результати (постумови):** користувачеві показано ранжований перелік оголошень-кандидатів; стан даних не змінено.

Сценарій 1.6 – Пропонування зв'язку між оголошеннями

Мета: запропонувати зв'язок між оголошенням про втрату та оголошенням про знахідку. **Основна діюча особа:** користувач (автор одного з оголошень). **Область дії:** вебзастосунок пошуку загублених тварин. **Передумови:** користувач автентифікований, є автором оголошення і переглядає перелік кандидатів. **Тригер:** користувач обирає кандидата і функцію «Запропонувати зв'язок». **Основний успішний сценарій:**

1. Користувач обирає кандидата зі свого переліку кандидатів.
2. Система перевіряє коректність пари (одне оголошення про втрату і одне про знахідку) та належність користувачу одного з оголошень.
3. Система перевіряє відсутність наявного зв'язку для цієї пари.
4. Система створює зв'язок у стані «запропоновано».
5. Друга сторона отримує запропонований зв'язок у переліку своїх зв'язків.

Розширення (альтернативні сценарії): 3а. Якщо для пари вже існує зв'язок, система відмовляє через обмеження унікальності. 2а. Якщо користувач не є автором жодного з оголошень пари, система відмовляє у доступі. **Результати (постумови):** створено зв'язок у стані «запропоновано», доступний другій стороні для розгляду.

Сценарій 1.7 – Підтвердження зв'язку та розкриття контактів

Мета: підтвердити збіг і отримати контактні дані другої сторони. **Основна діюча особа:** користувач (друга сторона зв'язку). **Область дії:** вебзастосунок пошуку загублених тварин. **Передумови:** для пари оголошень запропоновано зв'язок; користувач є автором другого оголошення. **Тригер:** користувач відкриває запропонований зв'язок. **Основний успішний сценарій:**

1. Користувач переглядає перелік своїх зв'язків.
2. Користувач обирає запропонований зв'язок.
3. Система перевіряє, що користувач не є ініціатором зв'язку.
4. Користувач підтверджує зв'язок.
5. Система переводить зв'язок у стан «підтверджено».
6. Система взаємно розкриває контактні дані обох оголошень.

Розширення (альтернативні сценарії): 3а. Якщо рішення намагається ухвалити ініціатор зв'язку, система відмовляє у доступі. 4а. Якщо користувач відхиляє зв'язок, система переводить його у стан «відхилено» і контакти не розкриває. **Результати (постумови):** зв'язок переведено у стан «підтверджено» і контактні дані обох сторін взаємно розкрито; у разі відхилення — стан «відхилено» без розкриття контактів.

Сценарій 1.8 – Зміна стану оголошення

Мета: перевести власне оголошення у завершальний стан. **Основна діюча особа:** користувач (автор оголошення). **Область дії:** вебзастосунок пошуку загублених тварин. **Передумови:** користувач автентифікований і є автором оголошення. **Тригер:** користувач на сторінці власного оголошення обирає зміну стану. **Основний успішний сценарій:**

1. Користувач відкриває власне оголошення.

2. Користувач обирає новий стан: для оголошення про втрату — «повернуто» або «закрито», для оголошення про знахідку — «вирішено» або «закрито».
3. Система перевіряє припустимість переходу.
4. Для переходу оголошення про втрату у стан «повернуто» система перевіряє наявність підтвердженого зв'язку.
5. Система зберігає новий стан.

Розширення (альтернативні сценарії): 4а. Якщо перехід у стан «повернуто» виконується без підтвердженого зв'язку, система блокує неприпустимий перехід. 3а. Якщо користувач не є автором оголошення, система відмовляє у доступі. **Результати (постумови):** оголошення збережено в новому припустимому стані; неприпустимі переходи не виконано.

Висновки до розділу 1

У першому розділі виконано аналіз предметної області пошуку загублених домашніх тварин. Показано, що наявні канали публікації оголошень розрізнені, оголошення не структуровані, а зіставлення втрати та знахідки виконується вручну, що збільшує час повернення тварини.

Проведено аналіз аналогів — сервісу PawBoost, бази Pet FBI та тематичних спільнот. Аналіз показав, що жодне з рішень не виконує автоматичного зіставлення оголошень про втрату та про знахідку, не ранжує ймовірні збіги і не приховує контактні дані до підтвердження збігу.

На основі аналізу сформовано функціональні та нефункціональні вимоги, вимоги до інтерфейсу користувача; функціональні вимоги визначено у формі восьми варіантів використання системи. Отримані вимоги є підставою для планування проєкту та проєктування системи в наступних розділах.

2 ПЛАНУВАННЯ ПРОЄКТУ З РОЗРОБКИ ВЕБЗАСТОСУНКУ

2.1 Визначення масштабу проєкту

Масштаб проєкту визначається переліком робіт за проєктом та їх результатами (deliverables). Розробку вебзастосунку поділено на вісім укрупнених етапів робіт, кожен з яких має визначений результат. Масштаб не ототожнюється з календарним планом: він задає, які роботи виконуються та що є їх результатом, тоді як терміни цих робіт визначає календарний план (підрозділ 2.3).

Перелік робіт за проєктом та їх результати:

- аналіз предметної області та аналогів — сформовані функціональні та нефункціональні вимоги;
- планування проєкту та оцінювання ризиків — оцінка розміру, календарний план і реєстр ризиків;
- розроблення алгоритму зіставлення та ранжування оголошень — специфікація алгоритму;
- проєктування моделей даних, архітектури та бази даних — проєктні моделі та схема бази даних;
- програмна реалізація серверної частини — реалізований прикладний програмний інтерфейс;
- програмна реалізація клієнтської частини — реалізований клієнтський застосунок;
- тестування функціональних і нефункціональних вимог — задокументовані результати тестування;
- підготовка експлуатаційного прикладу та оформлення роботи — готова кваліфікаційна робота.

Визначений перелік робіт є підставою для кількісної оцінки розміру проєкту (підрозділ 2.2), з якої далі виводиться календарний план робіт (підрозділ 2.3).

2.2 Оцінка розміру проекту

Розмір проекту оцінюється методом Use Case Points (UCP) на основі восьми варіантів використання, виокремлених у розділі вимог. Метод оцінює розмір за кількістю та складністю акторів і варіантів використання з урахуванням технічних і середовищних чинників.

Незведену вагу акторів (UAW) визначають за складністю взаємодії кожного актора із системою. Класифікацію акторів наведено в таблиці 2.1.

Таблиця 2.1 – Класифікація акторів

Актор	Тип взаємодії	Складність	Вага
Зареєстрований користувач	Графічний інтерфейс	Складний	3
Відвідувач публічної стрічки	Графічний інтерфейс	Складний	3
Зовнішнє файлове сховище	Програмний інтерфейс	Середній	2

Сумарна незведена вага акторів $UAW = 3 + 3 + 2 = 8$.

Незведену вагу варіантів використання (UUCW) визначають за складністю кожного варіанта: проста взаємодія (до 3 транзакцій) має вагу 5, середня (4–7 транзакцій) — вагу 10, складна (понад 7 транзакцій) — вагу 15. Розподіл восьми варіантів використання за складністю наведено в таблиці 2.2.

Таблиця 2.2 – Класифікація варіантів використання

Варіант використання	Складність	Вага
Реєстрація та вхід користувача	Проста	5
Перегляд і пошук публічної стрічки	Проста	5
Зміна стану оголошення	Проста	5
Публікація оголошення про загублену тварину	Середня	10
Публікація оголошення про знайдену тварину	Середня	10
Перегляд ранжованих кандидатів	Середня	10
Пропонування зв'язку між оголошеннями	Середня	10
Підтвердження або відхилення зв'язку	Середня	10

Сумарна незведена вага варіантів використання $UUCW = 3 \cdot 5 + 5 \cdot 10 = 65$.

Незведений показник UUCP обчислюють як суму незведених ваг акторів і варіантів використання згідно з формулою (2.1).

$$UUCP = UAW + UUCW \quad (2.1)$$

де UUCP — незведений показник Use Case Points; UAW — сумарна незведена вага акторів; UUCW — сумарна незведена вага варіантів використання.

Підстановка дає $UUCP = 8 + 65 = 73$.

Технічний коефіцієнт складності TCF враховує 13 технічних чинників: кожен оцінено за шкалою від 0 до 5 і помножено на відповідну вагу. Значення чинників і їх добутки наведено в таблиці 2.3.

Таблиця 2.3 – Технічні чинники складності

Чинник	Опис	Вага	Оцінка	Добуток
T1	Розподіленість системи	2	3	6
T2	Продуктивність	1	4	4
T3	Ефективність роботи кінцевого користувача	1	4	4
T4	Складність внутрішнього оброблення	1	3	3
T5	Повторна використовуваність коду	1	3	3
T6	Простота встановлення	0,5	3	1,5
T7	Простота використання	0,5	4	2
T8	Переносність	2	2	4
T9	Простота внесення змін	1	3	3
T10	Паралельність	1	2	2
T11	Безпека	1	4	4
T12	Доступ для сторонніх систем	1	1	1
T13	Потреба у навчанні користувачів	1	1	1

Сума добутків технічних чинників $\Sigma T = 38,5$. Технічний коефіцієнт складності обчислюють за формулою (2.2).

$$TCF = 0,6 + 0,01 \cdot \Sigma T \quad (2.2)$$

де TCF — технічний коефіцієнт складності; ΣT — сума добутків технічних чинників.

Підстановка дає $TCF = 0,6 + 0,01 \cdot 38,5 = 0,985$.

Середовищний коефіцієнт ECF враховує 8 чинників середовища розробки. Значення чинників і їх добутки наведено в таблиці 2.4.

Таблиця 2.4 – Середовищні чинники

Чинник	Опис	Вага	Оцінка	Добуток
E1	Обізнаність із процесом розробки	1,5	3	4,5
E2	Досвід у предметній області	0,5	3	1,5
E3	Досвід об'єктно-орієнтованої розробки	1	4	4
E4	Кваліфікація провідного аналітика	0,5	4	2
E5	Мотивація	1	5	5
E6	Стабільність вимог	2	4	8
E7	Часткова зайнятість персоналу	-1	0	0
E8	Складність мови програмування	-1	2	-2

Сума добутків середовищних чинників $\Sigma E = 23$. Середовищний коефіцієнт обчислюють за формулою (2.3).

$$ECF = 1,4 - 0,03 \cdot \Sigma E \quad (2.3)$$

де ECF — середовищний коефіцієнт; ΣE — сума добутків середовищних чинників.

Підстановка дає $ECF = 1,4 - 0,03 \cdot 23 = 0,71$.

Підсумковий розмір проекту в одиницях Use Case Points обчислюють як добуток незведеного показника на технічний і середовищний коефіцієнти згідно з формулою (2.4).

$$UCP = UUCP \cdot TCF \cdot ECF \quad (2.4)$$

де UCP — зведений показник Use Case Points.

Підстановка дає $UCP = 73 \cdot 0,985 \cdot 0,71 \approx 51$. При емпіричному коефіцієнті продуктивності 20 люд.-год на один UCP трудомісткість становить $51 \cdot 20 \approx 1020$ люд.-год, що за восьмигодинного робочого дня відповідає приблизно 128 люд.-дням.

Отримана оцінка розміру (≈ 51 UCP, ≈ 1020 люд.-год) є вихідною для побудови календарного плану робіт (підрозділ 2.3): орієнтовну трудомісткість розподілено між етапами робіт, визначеними в підрозділі 2.1.

2.3 Календарний план робіт

Календарний план робіт будується на основі оцінки розміру проекту (підрозділ 2.2): орієнтовну трудомісткість близько 1020 людино-годин (≈ 128 людино-днів для одного виконавця) розподілено між вісьмома етапами робіт, визначеними в підрозділі 2.1, з урахуванням їх послідовності та часткового суміщення в часі. Орієнтовний календарний план наведено в таблиці 2.5. Дати подано за стандартом ISO 8601.

Таблиця 2.5 – Календарний план робіт

Етап	Зміст робіт	Період	Тривалість, дн.
1	Аналіз предметної області, аналіз аналогів, формування вимог	2026-02-03 – 2026-02-12	10
2	Планування проекту, оцінювання ризиків	2026-02-13 – 2026-02-18	6
3	Розроблення алгоритму зіставлення та ранжування	2026-02-19 – 2026-02-27	9
4	Проектування моделей, архітектури, бази даних	2026-02-28 – 2026-03-08	9
5	Реалізація серверної частини	2026-03-09 – 2026-03-22	14
6	Реалізація клієнтської частини	2026-03-23 – 2026-04-03	12
7	Тестування функціональних і нефункціональних вимог	2026-04-04 – 2026-04-12	9
8	Експлуатаційний приклад, оформлення роботи	2026-04-13 – 2026-04-22	10

Загальна орієнтовна тривалість розробки одним виконавцем становить близько 79 робочих днів; вона менша за номінальні 128 людино-днів завдяки частковому суміщенню етапів у часі. Критичними є етапи 3–6, оскільки алгоритм зіставлення визначає вимоги до моделі даних, а модель даних — обсяг серверної та клієнтської реалізації. Затримка на цих етапах безпосередньо впливає на терміни тестування.

2.4 Оцінка ризиків

Оцінювання ризиків дозволяє завчасно визначити події, що можуть негативно вплинути на терміни або якість розробки, та передбачити заходи реагування. Для кожного ризику оцінено ймовірність і вплив за трирівневою шкалою (низький, середній, високий) та визначено захід реагування. Результати наведено в таблиці 2.6.

Таблиця 2.6 – Реєстр ризиків проекту

№	Ризик	Ймовірність	Вплив	Захід реагування
1	Неточність вимог до алгоритму зіставлення	Середня	Високий	Раннє прототипування алгоритму на тестових даних
2	Недооцінка трудомісткості серверної частини	Середня	Середній	Декомпозиція на модулі з визначеними межами
3	Хибні збіги через грубу геолокацію	Висока	Середній	Ранжування за відстанню і датою, підтвердження людиною
4	Витік контактних даних	Низька	Високий	Приховування контактів у публічних поданнях, розкриття лише після підтвердження
5	Циклічна залежність між модулями оголошень і зв'язків	Середня	Середній	Винесення спільного читання у листовий модуль
6	Зміна зовнішнього сховища файлів	Низька	Середній	Доступ до сховища через адаптер за портом
7	Некоректне введення гео-даних користувачем (наприклад, координати за межами України або очевидно недостовірні значення широти/довготи)	Висока	Середній	Валідація координат на стороні клієнта і сервера; обмеження допустимого діапазону широти 44°–53° пн.ш. і довготи 22°–41° сх.д.; візуальне підтвердження точки на мапі перед публікацією
8	Відмова зовнішнього сервісу хостингу (платформа PaaS, об'єктне сховище файлів)	Середня	Високий	Конфігурація доступу до сховища через адаптер за портом; зберігання резервних копій бази даних поза основним хостингом; план переходу на альтернативного провайдера з відновленням за добу

Найбільш значущими є ризики 1, 3, 4, 7 і 8. Ризик хибних збігів (ризик 3) знижується за рахунок ранжування кандидатів і обов'язкового підтвердження зв'язку людиною, що описано в розділі 4. Ризик витоку кон-

тактичних даних (ризик 4) знижується архітектурним рішенням про приховування контактів у публічних поданнях, що описано в розділах 3 і 4.

Висновки до розділу 2

У другому розділі виконано планування проєкту з розробки вебзастосунку. Масштаб проєкту визначено переліком робіт за проєктом та їх результатами — вісім укрупнених етапів робіт із визначеними результатами. Виконано оцінку розміру проєкту методом Use Case Points: зведений показник становить близько 51 UCP, що відповідає орієнтовній трудомісткості близько 1020 людино-годин. На основі оцінки розміру побудовано календарний план робіт із загальною тривалістю розробки близько 79 робочих днів та визначено критичні етапи, пов'язані з алгоритмом зіставлення, моделлю даних і реалізацією серверної частини.

Виконано оцінювання ризиків проєкту: складено реєстр із восьми ризиків, для кожного з яких оцінено ймовірність, вплив і визначено захід реагування. Найбільш значущі ризики пов'язані з точністю алгоритму, хибними збігами та захистом контактних даних; заходи реагування для них закладено в проєктні рішення наступних розділів.

3 ПРОЄКТУВАННЯ ВЕБЗАСТОСУНКУ

3.1 Проєктування моделі оголошення та зв'язку

Модель предметної області будується навколо трьох понять: користувач, оголошення та зв'язок. Користувач — це автентифікований обліковий запис; ролі «власник» і «особа, яка знайшла тварину» не зберігаються окремо, а визначаються відносно того, яке оголошення створив користувач.

Оголошення про втрату і оголошення про знахідку мають однаковий набір ознак, тому подаються однією сутністю з ознакою-дискримінатором «тип». Таке рішення зумовлене тим, що запит формування кандидатів перетинає обидва типи оголошень, і єдина сутність спрощує цей запит. Оголошення містить тип, посилання на автора, стан, вид тварини, необов'язкові текстові ознаки (порода, кличка, забарвлення, опис), посилання на фотографію, контактні дані, координати та дату події.

Стан оголошення змінюється за визначеними правилами переходів. Оголошення про втрату з активного стану може перейти у стан «повернуто» або «закрито»; оголошення про знахідку — у стан «вирішено» або «закрито». Перехід оголошення про втрату у стан «повернуто» дозволяється лише за наявності підтвердженого зв'язку. Неприпустимі переходи блокуються на рівні сервісу.

Зв'язок — це збережене, запропоноване людиною з'єднання рівно одного оголошення про втрату з рівно одним оголошенням про знахідку. Зв'язок має життєвий цикл «запропоновано» → «підтверджено» або «відхилено». Зв'язок пропонує користувач, який є автором одного з двох оголошень; підтверджує або відхиляє його лише автор другого оголошення. Для пари оголошень може існувати не більше одного зв'язку, що забезпечується обмеженням унікальності. Після підтвердження зв'язку контактні дані обох оголошень взаємно розкриваються.

3.2 Архітектура вебзастосунку

Систему організовано як багаторівневу (трирівневу) архітектуру і розгорнуто в монорепозіторії з клієнтським та серверним застосунками й спільними правилами. Виокремлено три рівні. Рівень подання — односторінковий клієнтський вебзастосунок, що відповідає за інтерфейс користувача. Рівень застосунку (бізнес-логіки) — серверний застосунок прикладного програмного інтерфейсу, що реалізує правила предметної області, авторизацію та зіставлення оголошень. Рівень даних — система керування базами даних PostgreSQL і зовнішнє файлове сховище фотографій. Рівень подання взаємодіє з рівнем застосунку через інтерфейс прикладного програмування на основі протоколу HTTP з обміном даними у форматі JSON; передавання фотографій виконується окремим бінарним потоком. Рівень застосунку звертається до рівня даних через репозіторії та порт файлового сховища. Отже, архітектура не зводиться до дволанкової «клієнт — сервер»: доступ до даних відокремлено в самостійний рівень, що узгоджує опис із чотирма складниками на рисунку 3.1 (клієнт, сервер, база даних, файлове сховище). Загальну архітектуру наведено на рисунку 3.1.

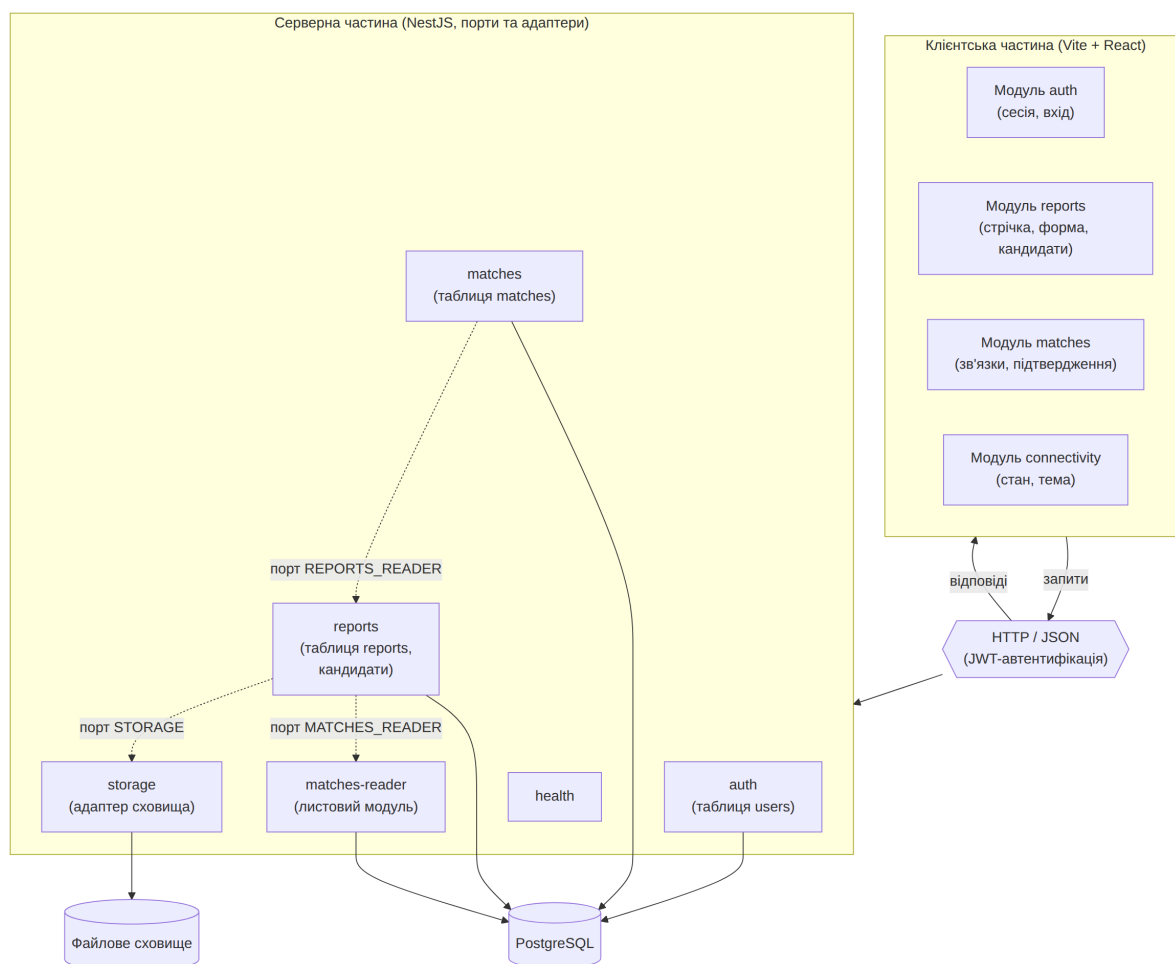


Рисунок 3.1 – Загальна архітектура вебзастосунку

Серверну частину організовано за принципом портів та адаптерів із фіксованим контрактом шарів. Контролер розбирає запит за схемою валідації, викликає рівно один метод сервісу і повертає результат. Сервіс містить бізнес-логіку та авторизацію і залежить від інших модулів лише через порти, задані символьними токенами. Репозиторій виконує запити до бази даних лише в межах таблиць власного модуля. Адаптер інкапсулює доступ до зовнішньої системи (наприклад, файлового сховища) за портом, що дозволяє замінити сховище без зміни бізнес-логіки.

Серверні модулі: модуль автентифікації (володіє таблицею користувачів), модуль оголошень (володіє таблицею оголошень, що об'єднує втрату і знахідку), модуль зв'язків, листовий модуль читання зв'язків, модуль файлового сховища, модуль діагностики стану та інфраструктурні модулі конфігурації, доступу до бази даних і спільного коду. Взаємну залежність модулів оголошень і зв'язків розв'язано винесенням спільно-

го читання підтверджених зв'язків у листовий модуль, що усуває циклічну залежність.

Клієнтську частину організовано за модулями з обов'язковим публічним інтерфейсом. Стан подається за схемою «розумний хук — нерозумний компонент»: уся логіка стану зосереджена в хуках, а компоненти лише відображають дані. Глобальне сховище стану не використовується; спільний стан передається через провайдери (сесія користувача, клієнт прикладного інтерфейсу, тема оформлення). Доступ до серверу інкапсульовано в єдиному транспортному модулі, який додає токен до кожного запиту, повторює запит один раз у разі простроченої сесії та повідомляє про завершення сесії.

Доступ до даних за замовчуванням заборонено: усі маршрути захищені глобальним фільтром автентифікації, а публічні маршрути позначаються явно. Канонічним ідентифікатором користувача є первинний ключ таблиці користувачів; усі зовнішні ключі посилаються на нього.

3.3 Проєктування структури бази даних

База даних реалізована у системі керування базами даних PostgreSQL [6] і складається з трьох таблиць: користувачі, оголошення та зв'язки. Структуру бази даних у вигляді діаграми «сутність — зв'язок» наведено на рисунку 3.2.

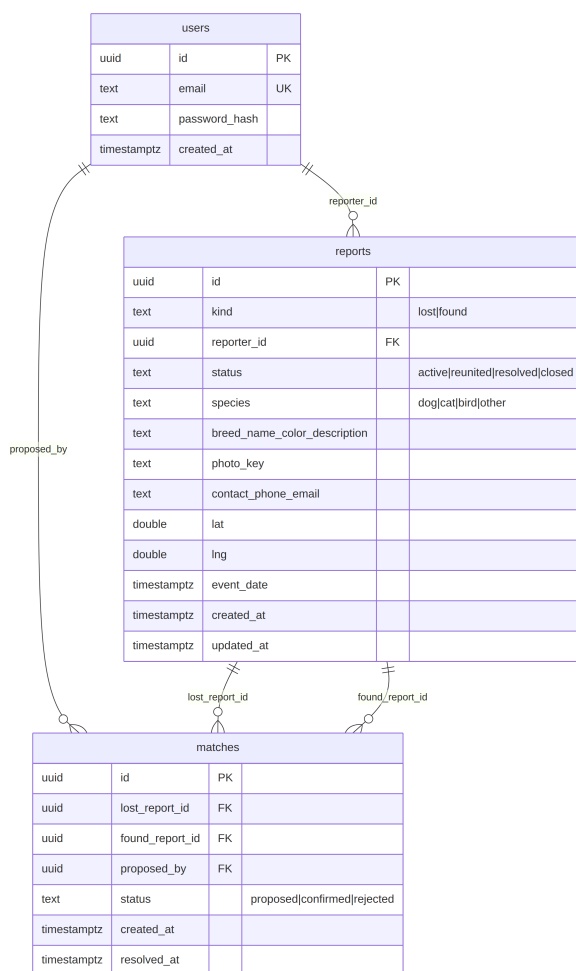


Рисунок 3.2 – Діаграма «сутність — зв'язок» бази даних

Структуру таблиці користувачів наведено в таблиці 3.1.

Таблиця 3.1 – Структура таблиці користувачів

Поле	Тип	Призначення
id	uuid	Первинний ключ, канонічний ідентифікатор користувача
email	text	Електронна пошта, унікальна
password_hash	text	Геш пароля за алгоритмом argon2id
created_at	timestampz	Дата та час створення запису

Структуру таблиці оголошень наведено в таблиці 3.2.

Таблиця 3.2 – Структура таблиці оголошень

Поле	Тип	Призначення
id	uuid	Первинний ключ оголошення
kind	text	Тип оголошення: втрата або знахідка
reporter_id	uuid	Зовнішній ключ на автора (користувачі)
status	text	Стан: активне, повернуто, вирішено, закрито
species	text	Вид тварини: собака, кіт, птах, інше
breed, name, color, description	text	Необов'язкові текстові ознаки
photo_key	text	Ключ фотографії у файловому сховищі
contact_phone, contact_email	text	Контактні дані (приховані у публічному поданні)
lat, lng	double precision	Координати місця події
event_date	timestampz	Дата та час події
created_at, updated_at	timestampz	Дата створення та оновлення запису

Структуру таблиці зв'язків наведено в таблиці 3.3.

Таблиця 3.3 – Структура таблиці зв'язків

Поле	Тип	Призначення
id	uuid	Первинний ключ зв'язку
lost_report_id	uuid	Зовнішній ключ на оголошення про втрату
found_report_id	uuid	Зовнішній ключ на оголошення про знахідку
proposed_by	uuid	Зовнішній ключ на користувача-ініціатора
status	text	Стан: запропоновано, підтверджено, відхилено
created_at	timestampz	Дата та час створення зв'язку
resolved_at	timestampz	Дата та час підтвердження або відхилення

Для таблиці зв'язків задано обмеження унікальності пари «оголошення про втрату — оголошення про знахідку», що гарантує не більше одного зв'язку для пари оголошень. Множинні значення (тип, стан, вид) зберігаються як текст і перевіряються переліком на межі застосунку, без використання перелічуваних типів бази даних, що спрощує подальше розширення набору значень.

3.4 Проектування на рівні класів

Серверну частину спроектовано як набір класів із фіксованим контрактом відповідальностей, що відповідає принципу портів та адаптерів (підрозділ 3.2). Діаграму класів серверної частини наведено на рисунку 3.3.

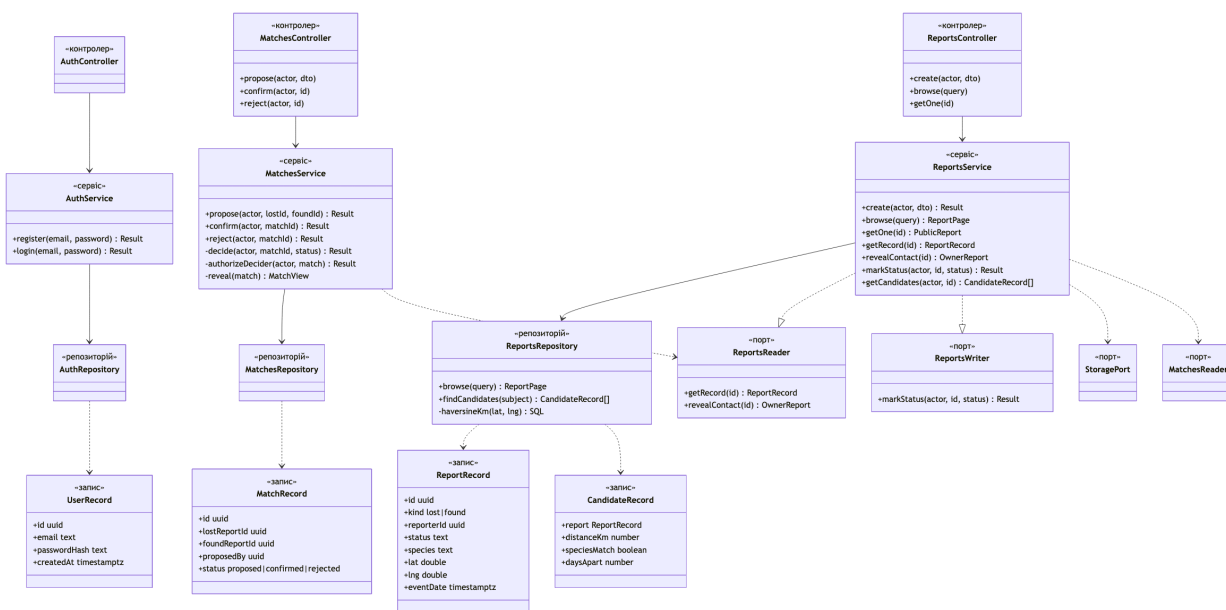


Рисунок 3.3 – Діаграма класів серверної частини

Класи розподілено за трьома шарами відповідальності: контролер — сервіс — репозитрій. Контролери AuthController, ReportsController та MatchesController розбирають запит за схемою валідації й делегують його обробку рівно одному методу відповідного сервісу, не містячи бізнес-логіки. Сервіси зосереджують бізнес-логіку та авторизацію. Репозитрії AuthRepository, ReportsRepository і MatchesRepository виконують запити лише до таблиць власного модуля, інкапсулюючи доступ до бази даних.

Предметну область подано класами-записами, узгодженими зі структурою таблиць (таблиці 3.1–3.3). Запис UserRecord описує користувача (ідентифікатор, електронна пошта, геш пароля). Запис ReportRecord описує оголошення (ідентифікатор, тип, автор, стан, вид тварини, обов'язкові текстові ознаки, ключ фотографії, контактні дані, координати, дата події). Запис MatchRecord описує зв'язок (ідентифікатор, посилання

на оголошення про втрату і про знахідку, ініціатор, стан, дати створення та розв'язання). Звужений запис кандидата `CandidateRecord` подає результат зіставлення й містить лише оголошення (`report`), відстань (`distanceKm`), збіг виду (`speciesMatch`) та різницю в днях (`daysApart`).

Відповідальність сервісів розподілено за модулями:

- `AuthService` — реєстрація та вхід користувача (методи `register` і `login`): геш пароля за алгоритмом `argon2id` і видача токена доступу;
- `ReportsService` — створення, перегляд переліку та отримання оголошення, розкриття контактних даних, зміна стану і формування переліку кандидатів (методи `create`, `browse`, `getOne`, `getRecord`, `revealContact`, `markStatus`, `getCandidates`), причому метод `markStatus` застосовує правила переходів стану, а метод `getCandidates` повертає ранжований перелік оголошень-кандидатів;
- `MatchesService` — пропонування, підтвердження та відхилення зв'язку (методи `propose`, `confirm`, `reject`); приватні методи `decide`, `authorizeDecider` і `reveal` реалізують спільну логіку рішення, перевірку того, що рішення ухвалює не ініціатор зв'язку, а автор другого оголошення, і взаємне розкриття контактних даних обох оголошень.

Власне зіставлення оголошень обчислюється на рівні репозиторію: метод `ReportsRepository.findCandidates` формує вибірку оголошень протилежного типу з показниками відповідності, а приватний метод `haversineKm` задає обчислення географічної відстані засобами бази даних. Сервіс `ReportsService` звертається до цього методу зі свого методу `getCandidates`; окремого сервісу зіставлення не виокремлено, оскільки зіставлення є частиною відповідальності модуля оголошень.

Залежності між модулями розв'язано через порти. Модуль зв'язків залежить від модуля оголошень не напряму, а через порт `ReportsReader` з методами `getRecord` і `revealContact`, який реалізує клас `ReportsService` (поряд із портом `ReportsWriter` для зміни стану). У зворотному напрямі модуль оголошень читає підтверджені зв'язки через порт `MatchesReader`, що його надає окремий листовий модуль `matches-reader`. Така схема усуває циклічну залежність між модулями оголошень і зв'язків, узгоджуючись з архітектурою (підрозділ 3.2). Доступ до файлового сховища інкап-

сульовано портом `StoragePort`, що дозволяє замінити сховище без зміни бізнес-логіки.

Діаграма класів безпосередньо відображає архітектуру портів та адаптерів (підрозділ 3.2): шари контролер — сервіс — репозиторій та порти `ReportsReader`, `ReportsWriter`, `MatchesReader` і `StoragePort` відповідають контракту шарів, а класи-записи `UserRecord`, `ReportRecord` і `MatchRecord` відповідають моделі предметної області з трьох понять (підрозділ 3.1).

Висновки до розділу 3

У третьому розділі спроектовано систему. Побудовано модель предметної області з трьох понять — користувач, оголошення, зв'язок; обґрунтовано подання оголошень про втрату і про знахідку єдиною сутністю з ознакою-дискримінатором та визначено правила переходів стану і життєвий цикл зв'язку.

Спроектовано архітектуру вебзастосунку як багаторівневу (трирівневу): рівень подання (клієнтський застосунок), рівень застосунку (серверний прикладний інтерфейс) і рівень даних (база даних і файлове сховище); серверну частину організовано за принципом портів та адаптерів із фіксованим контрактом шарів, клієнтську — за схемою «розумний хук — нерозумний компонент». Описано розв'язання взаємної залежності модулів оголошень і зв'язків.

Спроектовано структуру бази даних із трьох таблиць; для кожної таблиці визначено поля, типи та призначення, а також обмеження цілісності. Виконано проектування на рівні класів: визначено класи-записи, сервіси, репозиторії та порти. Отримані проектні рішення є основою програмної реалізації, описаної в наступному розділі.

4 ПРОГРАМНА РЕАЛІЗАЦІЯ ВЕБЗАСТОСУНКУ

4.1 Алгоритм зіставлення оголошень

Алгоритм зіставлення формує за заданим оголошенням ранжований перелік оголошень протилежного типу, які з найбільшою ймовірністю стосуються тієї самої тварини. Якщо вихідним є оголошення про втрату, кандидатами є оголошення про знахідку, і навпаки. Алгоритм ґрунтується на трьох ознаках, які піддаються формалізації та об'єктивному порівнянню: вид тварини, географічна відстань між місцями події та різниця дат події. Інші ознаки (порода, забарвлення, кличка, опис) подаються у вільній формі і використовуються людиною на етапі підтвердження, але в автоматичному ранжуванні не беруть участі, оскільки є суб'єктивними і часто неточними.

Робота алгоритму складається з таких етапів:

- формування ознак вихідного оголошення;
- попередня фільтрація множини оголошень-кандидатів;
- оцінювання відповідності кожного кандидата за відстанню, видом і різницею дат;
- ранжування кандидатів і формування впорядкованого переліку.

Перелік кандидатів не зберігається у базі даних, а обчислюється на запит. Остаточне рішення про збіг ухвалює людина, тому алгоритм виконує не класифікацію, а впорядкування кандидатів за спаданням ймовірності збігу.

Кожне оголошення подається набором формалізованих ознак: тип (втрата або знахідка); вид тварини (собака, кіт, птах, інше); географічні координати місця події (широта і довгота); дата та час події; стан (активне, повернуто, вирішено, закрито). Для зіставлення вихідного оголошення з кандидатом використовують його тип (для визначення протилежного типу), вид тварини, координати та дату події.

Попередня фільтрація зменшує множину оголошень, для яких виконується оцінювання відповідності, і відсікає свідомо нерелевантні записи за правилами: тип кандидата протилежний типу вихідного оголошення; кандидат не є вихідним оголошенням; за наявності обмеження

за радіусом (для пошуку у публічній стрічці) до перегляду потрапляють лише оголошення в межах заданого радіуса. Обмеження за радіусом є необов'язковим: якщо радіус не задано, у перелік потрапляють усі оголошення протилежного типу, а їх остаточне впорядкування виконується на етапі ранжування (підрозділ 4.2). Вид тварини на етапі фільтрації не відсікає кандидатів повністю, оскільки людина, яка знайшла тварину, може помилитися у визначенні виду; натомість вид враховується як пріоритетна ознака під час ранжування.

Для кожного кандидата, що пройшов попередню фільтрацію, обчислюються три показники відповідності: збіг за видом тварини, географічна відстань і різниця дат події. Збіг за видом є булевою ознакою: «істина», якщо вид тварини кандидата дорівнює виду вихідного оголошення, інакше «хиба».

Географічна відстань між місцями події обчислюється за формулою гаверсінуса як довжина дуги великого кола сфери [7]. Спершу обчислюється проміжна величина гаверсінуса центрального кута згідно з формулою (4.1):

$$a = \sin^2((\varphi_2 - \varphi_1) / 2) + \cos \varphi_1 \cdot \cos \varphi_2 \cdot \sin^2((\lambda_2 - \lambda_1) / 2) \quad (4.1)$$

де a — гаверсінус центрального кута між двома точками; φ_1, φ_2 — широти місць події першого та другого оголошень, рад; λ_1, λ_2 — довготи місць події першого та другого оголошень, рад.

Відстань між місцями події обчислюється згідно з формулою (4.2):

$$d = 2 \cdot R \cdot \arcsin(\sqrt{a}) \quad (4.2)$$

де d — відстань між місцями події, км; R — середній радіус Землі, $R = 6371$ км; a — гаверсінус центрального кута, обчислений за формулою (4.1).

Координати, задані у градусах, перед обчисленням переводяться у радіани. Менше значення відстані відповідає більшій географічній близькості і, відповідно, вищій ймовірності того, що оголошення стосуються тієї самої тварини. Різниця дат події є елементарним перетворенням —

модулем різниці позначок часу, нормованим до діб — і обчислюється згідно з формулою (4.3):

$$t = |T_2 - T_1| / 86400 \quad (4.3)$$

де t — різниця дат події, діб; T_1, T_2 — час події першого та другого оголошень, с; 86400 — кількість секунд у добі.

Менше значення різниці дат відповідає більшій узгодженості подій у часі. Усі три показники обчислюються безпосередньо засобами системи керування базами даних під час вибірки кандидатів, що дозволяє не завантажувати в пам'ять усю множину оголошень.

4.2 Ранжування кандидатів

Після обчислення показників відповідності кандидати впорядковуються. Для зведення трьох показників (збіг за видом s , географічна відстань d , різниця дат t) у єдиний порядок розглядалися два альтернативні підходи: формування єдиного зваженого скалярного рейтингу виду $R = w_1 \cdot s - w_2 \cdot d - w_3 \cdot t$ і лексикографічне впорядкування за пріоритетом ознак. Від зваженого підходу свідомо відмовлено з трьох причин: ваги $w_1 \dots w_3$ мали б бути визначені емпірично за наявності розмічених даних, яких на момент проєктування немає; нормалізація різнорідних величин (булеве значення, кілометри, доби) у спільний масштаб вносить методологічну непрозорість; єдине агреговане число приховує від користувача внутрішні складові оцінки і ускладнює пояснення, чому конкретний кандидат розташовано вище. Натомість обрано лексикографічне впорядкування, що відображає природний пріоритет ознак у предметній області: вид тварини є визначальним (різні види — з високою ймовірністю різні тварини), далі — географічна близькість (тварина рідко долає десятки кілометрів за добу), далі — узгодженість у часі.

Критеріями оцінювання у формулі (4.4) є три раніше обчислені показники, упорядковані за спаданням пріоритету:

- критерій першого рівня — збіг за видом тварини $s \in \{0; 1\}$; вища відповідність у кандидата з $s = 1$, тобто оголошення з тим самим ви-

дом має пріоритет над оголошенням з іншим видом незалежно від решти показників;

– критерій другого рівня — географічна відстань d , км; за рівного s вища відповідність у кандидата з меншим d ;

– критерій третього рівня — різниця дат події t , діб; за рівних s і d вища відповідність у кандидата з меншим t .

Сумарну відповідність визначають покроковим порівнянням критеріїв: для пари кандидатів x, y спершу порівнюють значення s ; якщо вони різні — порядок визначено і подальші ознаки не обчислюються; за рівних s порівнюють d ; за рівних d — t ; за повної рівності трьох показників порядок між x і y є невизначеним. Така схема відповідає лексикографічному відношенню переваги в багатокритеріальному аналізі рішень [8] і транслюється безпосередньо в речення `ORDER BY` бази даних без додаткових обчислень. Правило впорядкування формалізовано у формулі (4.4): для двох кандидатів x та y кандидат x передуює кандидату y , якщо

$$x < y \Leftrightarrow (s_x > s_y) \vee (s_x = s_y \wedge d_x < d_y) \vee (s_x = s_y \wedge d_x = d_y \wedge t_x < t_y), \quad (4.4)$$

де x, y — два довільні кандидати з множини попередньо відфільтрованих оголошень; $x < y$ — відношення « x передуює y в результаті ранжування»; s_x, s_y — збіг за видом тварини для кандидатів x, y (1 — істина, 0 — хиба); d_x, d_y — відстань між місцями події для кандидатів x, y , км, за формулою (4.2); t_x, t_y — різниця дат події для кандидатів x, y , діб, за формулою (4.3).

Згідно з формулою (4.4) спочатку вище розташовуються кандидати з тим самим видом тварини; серед них вище — географічно ближчі; за рівної відстані вище — кандидати з меншою різницею дат. Таке впорядкування є прозорим: користувач бачить для кожного кандидата конкретні значення відстані, збігу за видом і різниці дат, що полегшує ухвалення рішення про підтвердження зв'язку.

Розглянемо приклад. Нехай опубліковано оголошення про загублену тварину виду «собака» з координатами місця події (46,4825; 30,7233) і датою 2026-05-10. Є два оголошення про знахідку: кандидат А — вид «со-

бака», координати (46,4790; 30,7300), дата 2026-05-11; кандидат В — вид «собака», координати (46,5300; 30,6500), дата 2026-05-09. Для кандидата А відстань за формулами (4.1)–(4.2) становить близько 0,64 км, різниця дат за формулою (4.3) — 1 доба; для кандидата В відстань — близько 7,7 км, різниця дат — 1 доба. Обидва кандидати мають однаковий вид, тому $s_A = s_B = 1$, і згідно з формулою (4.4) визначальною стає відстань: оскільки $0,64 < 7,7$, кандидат А розташовується вище. Користувач отримує перелік, у якому першим наведено географічно найближче оголошення з тим самим видом тварини.

Покажемо обчислення відстані до кандидата А за формулами (4.1)–(4.2). Координати переводять у радіани; різниці широт і довгот становлять $\Delta\varphi = \varphi_A - \varphi_Q \approx -6,1 \cdot 10^{-5}$ рад і $\Delta\lambda = \lambda_A - \lambda_Q \approx 1,2 \cdot 10^{-4}$ рад, де індекс Q позначає вихідне оголошення про втрату. За формулою (4.1) проміжна величина $a = \sin^2(\Delta\varphi / 2) + \cos \varphi_Q \cdot \cos \varphi_A \cdot \sin^2(\Delta\lambda / 2) \approx 2,55 \cdot 10^{-9}$, а за формулою (4.2) відстань $d = 2 \cdot R \cdot \arcsin(\sqrt{a}) \approx 2 \cdot 6371 \cdot \arcsin(\sqrt{2,55 \cdot 10^{-9}}) \approx 0,64$ км, що збігається з наведеним вище значенням. Аналогічно для кандидата В відстань становить близько 7,7 км.

4.3 Вибір інструментів розробки

Вибір інструментів зумовлений вимогами до продуктивності, безпеки та підтримуваності, а також рішенням про монорепозиторій із клієнтською та серверною частинами на спільній мові програмування.

Серверну частину реалізовано на платформі Node.js [9] версії 22.11 LTS з використанням фреймворку NestJS [10] версії 11.0, який підтримує модульну структуру та впровадження залежностей, що відповідає обраній архітектурі портів та адаптерів. Доступ до бази даних виконано через ORM Drizzle [11] версії 0.44.2 з типобезпечними запитам; системою керування базами даних обрано PostgreSQL [6] версії 16.4. Валідацію вхідних даних виконано бібліотекою Zod [12] версії 3.23, обробку результатів операцій — бібліотекою neverthrow версії 8.1, що дозволяє повертати помилки як значення, а не винятки. Паролі гешуються алгоритмом argon2id (пакет `argon2` версії 0.41), автентифікація реалізована через підписаний токен JSON Web Token [13] (пакет `jsonwebtoken` версії 9.0).

Захист застосунку посилено типовими заголовками безпеки та обмеженням джерел запитів. Мовою реалізації обох частин є TypeScript версії 5.6.

Клієнтську частину реалізовано бібліотекою React [14] версії 19.0 із системою збирання Vite версії 6.0 та маршрутизацією React Router версії 6.28. Стилізацію виконано модулями CSS на основі єдиної системи візуальних токенів зі світлою та темною темами. Валідація відповідей сервера на клієнті також виконується бібліотекою Zod. Тестування реалізовано засобом Vitest версії 2.1; репозиторні тести серверної частини виконуються проти ефемерної бази даних PostgreSQL у контейнері засобами бібліотеки `testcontainers` версії 10.13, що забезпечує перевірку реальних запитів без імітації драйвера бази даних. Контейнеризація використовує Docker версії 27 з Docker Compose v2.

Монорепозиторій керується пакетним менеджером `pnpm` версії 10.0 і збирачем `Turbopack` версії 2.3, що дозволяє запускати єдиний конвеєр перевірки (перевірка типів, тести, збірка) для обох застосунків. Повний перелік версій із фіксацією точних номерів збережено у файлі ``pnpm-lock.yaml``, що забезпечує відтворюваність складання на будь-якому середовищі.

4.4 Реалізація сервісу зіставлення оголошень

Сервіс зіставлення реалізує алгоритм, описаний у підрозділах 4.1 і 4.2. Перелік кандидатів обчислюється на запит і не зберігається. Сервіс перевіряє, що запит надходить від автора оголошення, після чого репозиторій формує вибірку оголошень протилежного типу з обчисленням показників відповідності безпосередньо засобами бази даних.

Відстань обчислюється виразом мови SQL за математично еквівалентною формою — сферичною теоремою косинусів, яка дає ту саму довжину дуги великого кола, що й формули (4.1)–(4.2), але без обчислення половинних кутів; для уникнення похибки функції арккосинуса її аргумент обмежується відрізком від -1 до 1 . Різниця дат за формулою (4.3) обчислюється відніманням значень дати події з переведенням у доби; дата вихідного оголошення передається як рядок у форматі ISO 8601 із приведенням до типу позначки часу. Упорядкування виконується за пра-

вилом (4.4): спершу за збігом виду тварини за спаданням, далі за відстанню за зростанням, далі за різницею дат за зростанням. Спрощену реалізацію формування кандидатів наведено в лістингу 4.1.

Лістинг 4.1 – Спрощена реалізація формування переліку кандидатів

```
findCandidates(subject: ReportRecord): ResultAsync<CandidateRecord[], DbError> {
  const oppositeKind = subject.kind === 'lost' ? 'found' : 'lost';
  const distanceKm = this.haversineKm(subject.lat, subject.lng);
  const speciesMatch = sql<boolean>`${reports.species} = ${subject.species}`;
  const subjectDateIso = subject.eventDate.toISOString();
  const daysApart = sql<number>`abs(extract(epoch from
    (${reports.eventDate} - ${subjectDateIso}::timestampz)) / 86400)`;

  const query = this.db
    .select({ row: reports, distanceKm, speciesMatch, daysApart })
    .from(reports)
    .where(and(eq(reports.kind, oppositeKind), ne(reports.id, subject.id)))
    .orderBy(desc(speciesMatch), distanceKm, daysApart);

  return ResultAsync.fromPromise(query, (cause) => dbError(cause))
    .map((rows) => rows.map(toCandidate));
}
```

Підтвердження зв'язку реалізовано в окремому сервісі. Під час пропонування зв'язку перевіряється коректність пари (одне оголошення про втрату і одне про знахідку) та належність ініціатору одного з оголошень. Підтвердити або відхилити зв'язок може лише автор другого оголошення; спроба ініціатора ухвалити рішення відхиляється. Після підтвердження зв'язку контактні дані обох оголошень взаємно розкриваються, при цьому модуль зв'язків не звертається до таблиці оголошень безпосередньо, а отримує дані через порт модуля оголошень. Повний код схеми бази даних (лістинг A.1), сервісу зіставлення (лістинг A.2), обчислення відстані (лістинг A.3) та сервісу підтвердження зв'язку (лістинг A.4) наведено в додатку A.

Висновки до розділу 4

У четвертому розділі описано програмну реалізацію вебзастосунку. Розроблено алгоритм зіставлення оголошень: визначено набір формалізованих ознак, етапи роботи (формування ознак, попередня фільтрація, оцінювання, ранжування), обґрунтовано обчислення географічної відстані за формулою гаверсінуса (формули (4.1)–(4.2)), різниці дат (формула (4.3)) та булевого збігу за видом. Обґрунтовано лексикографічне правило ранжування (формула (4.4)) як прозоре і таке, що відображає

природний пріоритет ознак; наведений приклад підтверджує коректність впорядкування.

Обґрунтовано вибір інструментів розробки: платформа Node.js, фреймворк NestJS, бібліотека React, ORM Drizzle, система керування базами даних PostgreSQL, бібліотеки валідації та оброблення помилок, монорепозиторій під керуванням рnpm і Turborepo. Описано реалізацію сервісу зіставлення, що обчислює показники відповідності та впорядковує кандидатів засобами бази даних за лексикографічним правилом, а також реалізацію підтвердження зв'язку із взаємним розкриттям контактних даних і дотриманням меж модулів. Повні лістинги винесено в додаток А. Ефективність розробленого зіставлення перевіряється експериментально (розділ 6), а коректність — тестуванням (розділ 5).

5 ТЕСТУВАННЯ ВЕБЗАСТОСУНКУ

5.1 Процес тестування

Тестування організовано як автоматизований процес, інтегрований у єдиний конвеєр перевірки (перевірка типів, тести, збірка), що виконується для обох застосунків монорепозиторію. Застосовано три рівні тестування серверної частини та окремий рівень клієнтської. На модульному рівні перевіряють сервіси та чисті функції з бізнес-логікою та авторизацією, ізолюючи залежності підставними реалізаціями портів. На інтеграційному рівні перевіряють репозиторії проти реальної ефемерної бази даних PostgreSQL, що розгортається в контейнері засобами бібліотеки testcontainers, без імітації драйвера. На системному (наскрізному) рівні перевіряють контролери, які виконують реальні запити крізь увесь конвеєр оброблення. Клієнтську частину перевіряють на рівні хуків і компонентів з імітацією мережевих запитів.

Стратегію тестування переважно автоматизовано; ручну перевірку застосовано до зручності інтерфейсу та мобільної придатності. Критерієм проходження є успішне виконання всіх тестів і завершення конвеєра перевірки без помилок. Розподіл рівнів тестування за об'єктом перевірки та засобами наведено в таблиці 5.1.

Таблиця 5.1 – Рівні та засоби тестування

Рівень	Об'єкт перевірки	Засоби
Модульне	Сервіси та чисті функції (бізнес-логіка, авторизація)	Vitest, підставні реалізації портів
Інтеграційне	Репозиторії проти реальної бази даних	Vitest, ефемерна PostgreSQL у контейнері (testcontainers)
Системне (наскрізне)	Контролери крізь увесь конвеєр оброблення запиту	Vitest, реальні HTTP-запити
Клієнтське	Хуки та компоненти клієнтської частини	Vitest, імітація мережевих запитів

Тестовим середовищем є локальний конвеєр на платформі Node.js із засобом Vitest; інтеграційні тести використовують контейнеризовану PostgreSQL (Docker), що забезпечує відтворюваність перевірок на будь-

якому середовищі. Загалом автоматизований набір містить 130 серверних і 44 клієнтських тести; усі тести проходять успішно.

5.2 Функціональне тестування

Метою функціонального тестування є перевірка відповідності реалізованої системи вимогам, сформованим у підрозділі 1.3. Перевірку виконано за тест-кейсами: кожен задає передумови, послідовність дій та очікуваний результат, а фактичний результат зафіксовано за прогоном автоматизованого набору. Перелік застосованих тест-кейсів і результати наведено в таблиці 5.2.

Таблиця 5.2 – Тест-кейси функціонального тестування

ТК	Передумови	Кроки	Очікуваний результат	Фактичний результат
ТК-1	Користувач не зареєстрований	Ввести унікальну пошту і пароль, підтвердити	Створено обліковий запис, видано токен	Пройдено
ТК-2	Пошта вже зареєстрована	Ввести зайняту пошту, підтвердити	Відмова «пошту зайнято», запис не створено	Пройдено
ТК-3	Обліковий запис існує	Ввести правильні пошту і пароль	Видано токен і дані користувача	Пройдено
ТК-4	Обліковий запис існує	Ввести правильну пошту і невірний пароль	Відмова «невірні дані»	Пройдено
ТК-5	Користувач автентифікований	Заповнити форму коректними даними, підтвердити	Оголошення збережено зі станом «активне»	Пройдено
ТК-6	Користувач автентифікований	Подати форму з порожніми обов'язковими полями	Відмова з повідомленням про помилку, запис не створено	Пройдено
ТК-7	Є активні оголошення	Відкрити стрічку, задати фільтри виду й періоду	Повернено відфільтровану сторінку без контактів	Пройдено
ТК-8	Користувач — автор оголошення	Відкрити власне оголошення, запросити кандидатів	Повернено ранжований перелік кандидатів	Пройдено
ТК-9	Користувач не є автором оголошення	Запросити кандидатів чужого оголошення	Відмову у доступі	Пройдено
ТК-10	Користувач — автор, переглядає кандидатів	Обрати кандидата, «Запропонувати зв'язок»	Зв'язок створено у стані «запропоновано»	Пройдено
ТК-11	Зв'язок запропоновано, актор — друга сторона	Відкрити зв'язок, підтвердити	Зв'язок «підтверджено», контакти взаємно розкрито	Пройдено
ТК-12	Зв'язок запропоновано, актор — ініціатор	Спробувати підтвердити власний зв'язок	Відмову у доступі	Пройдено
ТК-13	Оголошення про втрату без підтвердженого зв'язку	Перевести оголошення у стан «повернуто»	Відмова: неприпустимий перехід	Пройдено
ТК-14	Для пари оголошень уже існує зв'язок	Запропонувати ще один зв'язок для тієї самої пари	Відмова: конфлікт унікальності	Пройдено

Усі тест-кейси пройдено. Єдиний конвеєр перевірки (перевірка типів, тести, збірка) для обох застосунків завершується без помилок.

5.3 Тестування нефункціональних вимог

Нефункціональне тестування перевіряє відповідність системи вимогам, сформованим у підрозділі 1.4.

Продуктивність. Формування переліку кандидатів виконується одним запитом до бази даних з обчисленням показників відповідності на боці бази даних, без завантаження всієї множини оголошень у пам'ять. Фільтрація і впорядкування виконуються засобами бази даних, що зберігає продуктивність зі зростанням обсягу даних. Кількісну перевірку часу формування переліку за обсягу до десятків тисяч оголошень і відповідність порогу у 2 секунди наведено в розділі 6.

Безпека. Паролі зберігаються лише у вигляді геш-значень за алгоритмом `argon2id`. Доступ до даних за замовчуванням заборонено: усі маршрути захищені глобальним фільтром автентифікації, публічні маршрути позначені явно. Контактні дані відсутні в публічних поданнях і розкриваються лише після підтвердження зв'язку. Перевірено, що публічні подання оголошень не містять контактних полів, а спроби несанкціонованого доступу відхиляються з відповідним кодом.

Надійність. Некоректні вхідні дані відхиляються на межі системи з типізованим повідомленням про помилку. Неприпустимі переходи стану оголошення блокуються. Обмеження унікальності пари оголошень запобігає створенню повторних зв'язків навіть за одночасних запитів. Тести контролерів перевіряють як успішний шлях, так і кожну категорію помилки.

Зручність використання. Інтерфейс перевірено на придатність до роботи з мобільного пристрою; стан оголошення подається одночасно піктограмою, підписом і кольором, що забезпечує доступність для користувачів із порушеннями сприйняття кольору.

Підсумок перевірки нефункціональних вимог підрозділу 1.4 за способом перевірки та результатом наведено в таблиці 5.3.

Таблиця 5.3 – Перевірка нефункціональних вимог

Нефункціональна вимога	Спосіб перевірки	Результат
Продуктивність (≤ 2 с)	Вимірювання часу формування переліку (розділ 6)	Виконано: десятки мілісекунд за 50 000 оголошень
Безпека	Тести публічних подань і не-санкціонованого доступу	Виконано: контакти приховано, доступ відхилено
Надійність	Тести меж системи та переходів стану	Виконано: некоректні дані й переходи блокуються
Масштабованість	Заміна файлового сховища через адаптер за портом	Виконано: сховище замінне без зміни бізнес-логіки
Підтримуваність	Модульна структура з визначеними межами	Виконано: модулі з явним публічним інтерфейсом
Зручність використання	Перевірка мобільної придатності та доступності	Виконано: стан подано піктограмою, підписом і кольором
Цілісність даних	Тест обмеження унікальності пари оголошень	Виконано: повторний зв'язок відхилено

Висновки до розділу 5

У п'ятому розділі виконано тестування вебзастосунку. Функціональне тестування підтвердило відповідність системи вимогам підрозділу 1.3: перевірено реєстрацію та вхід, публікацію оголошень, перегляд стрічки і кандидатів, пропонування та підтвердження зв'язку, правила переходів стану і обмеження цілісності. Автоматизований набір із 130 серверних і 44 клієнтських тестів проходить успішно.

Нефункціональне тестування підтвердило відповідність вимогам підрозділу 1.4 за безпекою, надійністю та зручністю використання; кількісну перевірку продуктивності винесено в розділ 6. Результати тестування свідчать про коректність роботи системи; кількісне дослідження ефективності зіставлення наведено в наступному розділі.

6 ЕКСПЕРИМЕНТАЛЬНЕ ДОСЛІДЖЕННЯ ЕФЕКТИВНОСТІ ЗІСТАВЛЕННЯ

6.1 Методика експерименту та модельний набір даних

Мету роботи сформульовано як підвищення ефективності первинного зіставлення оголошень, і ефективність визначено через вимірні величини: метрики ранжування (частку влучень істинного збігу в перші K позицій $\text{Hit}@K$ і середній обернений ранг MRR) та час формування переліку кандидатів (вступ). Цей розділ кількісно перевіряє досягнення мети на модельному наборі даних з відомими істинними збігами.

Набір даних згенеровано програмно за фіксованим початковим значенням генератора псевдовипадкових чисел, тому експеримент є відтворюваним. Координати оголошень лежать у межах України (широта від 44° до 53° пн.ш., довгота від 22° до 41° сх.д., узгоджено з обмеженням ризику 7 у підрозділі 2.3) і згруповані навколо десяти міських центрів, що відтворює реальну просторову кластеризацію оголошень. Набір містить 60 пар «втрата — знахідка» з відомим істинним збігом: для кожної пари оголошення про знахідку розташовано в межах близько 1–2 км і 0–2 діб від відповідного оголошення про втрату, що відповідає поведінці тварини у перші дні після зникнення. Додатково згенеровано 140 сторонніх оголошень про знахідку з різними видами, місцями і датами, які створюють конкуренцію істинним збігам під час ранжування. Загалом набір налічує 200 оголошень про знахідку, кожне з яких є кандидатом для будь-якого з 60 запитів.

Для кожного оголошення про втрату обчислюється ранжований перелік усіх оголошень про знахідку за правилом (4.4): спершу за збігом виду, далі за географічною відстанню (формули (4.1)–(4.2)), далі за різницею дат (формула (4.3)). У впорядкованому переліку визначається позиція (ранг) істинного збігу. Оцінювальний скрипт відтворює те саме правило ранжування, що й сервіс зіставлення (підрозділ 4.4), і зберігається разом із роботою (``thesis/experiment/run_experiment.py``).

Метрики ранжування обчислюють за трьома показниками. Показник $\text{Hit}@K$ — частка запитів, для яких істинний збіг потрапив у перші K

позицій переліку; обчислено для $K = 1, 3, 5$. Середній обернений ранг (MRR) — середнє значення оберненого рангу істинного збігу за всіма запитамі; він дорівнює одиниці, якщо істинний збіг завжди перший, і спадає зі збільшенням рангу. Додатково наведено медіанний ранг істинного збігу.

6.2 Оцінювання метрик ранжування

Результати оцінювання метрик ранжування на модельному наборі наведено в таблиці 6.1.

Таблиця 6.1 – Метрики ранжування (60 запитів, 200 оголошень-кандидатів)

Показник	Значення
Hit@1	0,82
Hit@3	1,00
Hit@5	1,00
Середній обернений ранг (MRR)	0,91
Медіанний ранг істинного збігу	1

Істинний збіг потрапляє на першу позицію у 82 % запитів і в усіх без винятку запитах — у перші три позиції (Hit@3 = 1,00). Середній обернений ранг 0,91 і медіанний ранг 1 свідчать, що в типовому випадку істинний збіг є першим або другим кандидатом у переліку. Випадки, коли істинний збіг не перший, зумовлені сторонніми оголошеннями того самого виду, розташованими ближче до місця події, ніж істинний збіг; такі кандидати закономірно ранжуються вище за відстанню, а остаточне рішення ухвалює людина, переглянувши перші позиції переліку.

Розподіл позицій (рангів) істинного збігу за всіма 60 запитами наведено в таблиці 6.2.

Таблиця 6.2 – Розподіл позицій істинного збігу в ранжованому переліку

Ранг істинного збігу	Кількість запитів	Частка, %
1	49	82
2	10	17
3	1	2

Розподіл підтверджує, що у переважній більшості запитів (82 %) істинний збіг є першим кандидатом переліку, а нижче за третю позицію він не опускається в жодному запиті. Отже, для ухвалення рішення користувачеві достатньо переглянути лише перші позиції ранжованого переліку, тоді як за неструктурованого перегляду відповідне оголошення могло б опинитися на будь-якій позиції серед сотень дописів.

6.3 Порівняння з базовою лінією та час формування переліку

Для оцінювання практичного ефекту ранжування його порівняно з базовою лінією — хронологічним переглядом оголошень про знахідку без ранжування, який власник виконував би у стрічці вручну. За хронологічного перегляду позиція істинного збігу у переліку є фактично випадковою, тому очікувана медіанна позиція дорівнює приблизно половині довжини переліку: для 200 оголошень — близько 100. Ранжування зміщує медіанну позицію істинного збігу до 1. Відношення цих величин дає скорочення обсягу перегляду приблизно у 100 разів: замість перегляду в середньому сотні оголошень користувач знаходить істинний збіг серед перших позицій ранжованого переліку.

Час формування ранжованого переліку виміряно для наборів із 1 000, 10 000 і 50 000 оголошень за допомогою еталонної реалізації правила ранжування, що відтворює обчислення показників відповідності та лексикографічне впорядкування (підрозділи 4.1–4.2). Вимірювання оцінює алгоритмічну вартість формування переліку, яка визначається обчисленням трьох показників для кожного кандидата та впорядкуванням зі складністю $O(n \log n)$. Результати наведено в таблиці 6.3.

Таблиця 6.3 – Час формування ранжованого переліку залежно від кількості оголошень

Кількість оголошень	Час формування переліку, мс
1 000	≈ 1
10 000	≈ 9
50 000	≈ 49

Час зростає приблизно лінійно з кількістю оголошень і для 50 000 записів не перевищує 50 мс, що значно менше за нефункціональну вимогу у 2 секунди (підрозділ 1.4). Оскільки у промисловій реалізації обчислення показників і впорядкування виконуються на боці бази даних з використанням індексів, наведені значення є консервативною верхньою оцінкою; реальний час відповіді сервісу залишається в межах вимоги навіть за обсягу в десятки тисяч активних оголошень.

Висновки до розділу 6

У шостому розділі експериментально перевірено досягнення мети роботи на відтворюваному модельному наборі даних із відомими істинними збігами. Метрики ранжування високі: істинний збіг потрапляє у перші три позиції переліку в усіх запитах ($\text{Hit}@3 = 1,00$), у перші п'ять — теж в усіх ($\text{Hit}@5 = 1,00$), на першу позицію — у 82 % запитів ($\text{Hit}@1 = 0,82$), а середній обернений ранг становить 0,91 за медіанного рангу істинного збігу, що дорівнює 1.

Порівняння з базовою лінією показало, що ранжування зменшує обсяг перегляду оголошень приблизно у 100 разів порівняно з хронологічним переглядом. Час формування ранжованого переліку для 50 000 оголошень не перевищує 50 мс, що відповідає нефункціональній вимозі у 2 секунди із значним запасом.

Отримані числові результати підтверджують досягнення поставленої мети: розроблений вебзастосунок підвищує ефективність первинного зіставлення оголошень — забезпечує високі метрики ранжування (істинний збіг серед перших позицій) і формує перелік кандидатів за час, що задовольняє вимогу продуктивності.

7 ЕКСПЛУАТАЦІЯ ЗАСТОСУНКУ

7.1 Робота користувача з системою

Робота користувача із системою починається з реєстрації або входу. Після автентифікації користувач отримує доступ до публікації оголошень, перегляду власних оголошень і зв'язків. Головну сторінку та публічну стрічку оголошень наведено на рисунку 7.1.

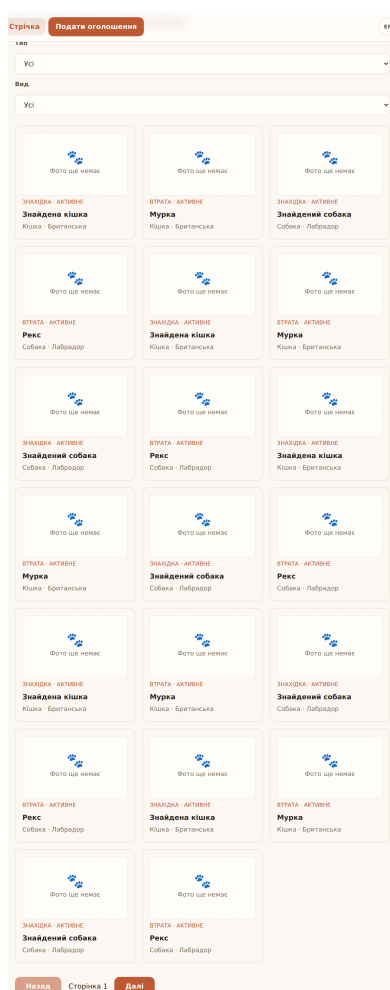


Рисунок 7.1 – Публічна стрічка оголошень

Публічна стрічка доступна без автентифікації і не містить контактних даних. Стрічка підтримує фільтрацію за видом тварини, станом оголошення, місцем і періодом події. Для кожного оголошення відображаються вид тварини, місце та дата події, стан, а також фотографія за її наявності.

Для публікації оголошення користувач обирає тип (втрата або знахідка) і заповнює форму. Форму публікації оголошення наведено на рисунку 7.2.

Рисунок 7.2 – Форма публікації оголошення

Форма містить послідовні секції: вид тварини та опис, місце та дата події, контактні дані, необов'язковий крок долучення фотографії. Якщо завантаження фотографії не вдалося, оголошення все одно зберігається, а користувача повідомляють про це; такий підхід зменшує ймовірність втрати оголошення через тимчасову помилку.

Для власного оголошення користувач відкриває сторінку кандидатів. Сторінку кандидатів наведено на рисунку 7.3.

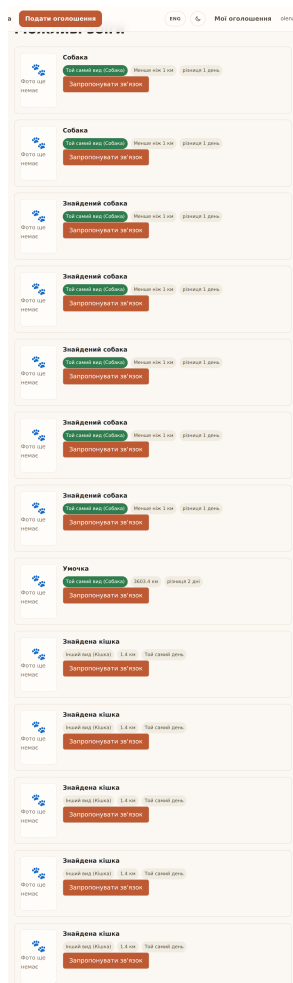


Рисунок 7.3 – Сторінка перегляду кандидатів

Для кожного кандидата відображаються відстань у кілометрах, ознака збігу за видом тварини та різниця дат події. Перелік упорядковано за правилом ранжування з розділу 4. Користувач може запропонувати зв'язок між своїм оголошенням і обраним кандидатом. Друга сторона отримує запропонований зв'язок у переліку своїх зв'язків і підтверджує або відхиляє його. Після підтвердження контактні дані обох сторін взаємно розкриваються.

7.2 Приклад зіставлення оголошень про загублену тварину

Розглянемо наскрізний приклад. Власник публікує оголошення про загублену тварину виду «собака» з місцем події у центрі міста і датою 2026-05-10. Незалежно від нього інша людина публікує оголошення про знайдену тварину виду «собака» неподалік із датою 2026-05-11.

Власник відкриває сторінку кандидатів свого оголошення. Система формує перелік активних оголошень про знахідку, обчислює для кожного відстань за формулами (4.1)–(4.2), різницю дат за формулою (4.3) і збіг за видом, після чого впорядковує перелік за правилом (4.4). Оголошення про знайдену тварину того самого виду на відстані близько 0,6 км з різницею в одну добу опиняється на першій позиції переліку.

Власник переглядає кандидата, пересвідчується у схожості за неформалізованими ознаками (порода, забарвлення, фотографія) і пропонує зв'язок. Друга сторона отримує запропонований зв'язок, підтверджує його, після чого система розкриває контактні дані обох сторін. Сторони зв'язуються та домовляються про повернення тварини. Після повернення власник переводить оголошення про втрату у стан «повернуто»; система дозволяє цей перехід, оскільки для оголошення існує підтверджений зв'язок. Підсумкову сторінку підтвердженого зв'язку наведено на рисунку 7.4.

Зв'язок

Стан: **Підтверджено**

Контактні дані розкрито

Ці дані показано, оскільки зв'язок підтверджено. В усіх інших місцях вони лишаються прихованими.

Оголошення про втрату: Мурка

Телефон: +380501119988

Пошта: olena.owner@petfinder.example

Оголошення про знахідку: Знайдена кішка

Телефон: +380677654321

Пошта: —

Рисунок 7.4 – Сторінка підтвердженого зв'язку з розкритими контактами

Приклад демонструє, що система автоматизує найбільш трудомісткий етап — виявлення ймовірного збігу — і зберігає за людиною остаточне рішення, що знижує ризик хибних збігів.

7.3 Розгортання застосунку адміністратором (розробником)

На відміну від попередніх підрозділів, що описують роботу кінцевого користувача, цей підрозділ призначений для адміністратора або розробника, який розгортає систему на цільовому середовищі (локальна машина, тестовий стенд, продуктивне середовище). Розгортання вебзастосунку складається з чотирьох послідовних кроків: підготовка робочого оточення, конфігурація змінних середовища, ініціалізація бази даних і запуск контейнерів.

Підготовка робочого оточення передбачає встановлення інструментів, узгоджених у підрозділі 4.3: Node.js версії 22 LTS, менеджера пакетів `pnpm` версії 10.x, середовища контейнеризації Docker (з підтримкою Docker Compose). Вихідний код одержують клонуванням репозиторію проекту (`git clone ...`) та встановленням залежностей командою `pnpm install` із кореня монорепозиторію.

Конфігурація здійснюється через файли змінних середовища `apps/server/.env` (для серверної частини) та `apps/client/.env` (для клієнтської частини). Обов'язковими змінними серверної частини є:

- `DATABASE_URL` — URL підключення до PostgreSQL у форматі `postgresql://user:password@host:port/dbname`;
- `PORT` — TCP-порт, на якому слухає серверна частина (за замовчуванням 3001);
- `JWT_SECRET` — секрет для підпису токенів автентифікації, не менше 32 символів;

–

`STORAGE_DIR` — шлях до каталогу локального файлового сховища, або параметри підключення до зовнішнього об'єктного сховища (через адаптер за портом, див. розділ 3);

- `CORS_ORIGIN` — перелік дозволених джерел запитів від клієнтської частини.

Обов'язковою змінною клієнтської частини є `VITE_API_URL` — повний URL серверного API, доступний браузеру кінцевого користувача. Усі секрети не комітяться у систему контролю версій; для прикладу значень використовується файл `.env.example`, що зберігається у репозиторії.

Ініціалізація бази даних виконується у два кроки. По-перше, створюється порожня база даних PostgreSQL (локально — командою `createdb`, у контейнері — автоматично під час старту відповідного сервісу). По-друге, виконуються міграції схеми командою `pnpm --filter @diplomna-robota/server db:migrate`, що використовує систему міграцій Drizzle ORM: інструмент послідовно застосовує SQL-файли з каталогу `apps/server/drizzle/`, фіксуючи виконані міграції у службовій таблиці `__drizzle_migrations`. За потреби наповнення тестовими даними виконують команду `pnpm --filter @diplomna-robota/server db:seed`.

Запуск контейнерів виконується командою `docker compose up -d` із кореня репозиторію. Файл `docker-compose.yml` оголошує три сервіси: `postgres` — база даних з томом для довгострокового збереження; `server` — серверна частина на Node.js, що читає змінні з `apps/server/.env`; `client` — клієнтська частина, попередньо зібрана командою `pnpm --filter @diplomna-robota/client build` і обслуговувана статичним сервером. Перевірка успішного запуску виконується запитом до перевірного маршруту серверної частини (`GET /healthz`), який має повернути код 200 та поточну версію застосунку. У продуктивному середовищі перед `server` і `client` рекомендовано розташувати зворотний проксі (наприклад, Nginx або Caddy) із підтримкою HTTPS і автоматичним поновленням сертифікатів.

Зупинка системи виконується командою `docker compose down`; том бази даних при цьому зберігається. Резервне копіювання бази даних виконується утилітою `pg_dump` за окремим розкладом, що визначається політикою адміністратора, з урахуванням ризику № 8 з реєстру ризиків (підрозділ 2.3).

Висновки до розділу 7

У цьому розділі розглянуто експлуатацію розробленого вебзастосунку. Описано роботу користувача із системою: реєстрацію та вхід, перегляд публічної стрічки з фільтрами, публікацію оголошення з обов'язковим долученням фотографії, перегляд ранжованих кандидатів і роботу зі зв'язками.

Наведено наскрізний приклад зіставлення оголошення про загублену тварину з оголошенням про знахідку: від публікації до підтвердження зв'язку, розкриття контактів і переведення оголошення у стан «повернуто». Приклад підтвердив, що система автоматизує виявлення ймовірного збігу, зберігаючи остаточне рішення за людиною, і коректно застосовує правила переходів стану.

Описано порядок розгортання системи адміністратором або розробником: підготовка робочого оточення (Node.js, npm, Docker), конфігурація змінних середовища серверної та клієнтської частин, ініціалізація бази даних із застосуванням міграцій Drizzle ORM, запуск контейнерів засобами Docker Compose і перевірка працездатності за маршрутом `/healthz``.

ЗАГАЛЬНІ ВИСНОВКИ

У кваліфікаційній роботі досягнуто мети — підвищено ефективність первинного зіставлення оголошень про загублених і знайдених домашніх тварин шляхом розроблення вебзастосунку, що автоматично формує ранжований перелік ймовірних збігів. Розв'язано задачу розроблення програмних засобів для автоматичного зіставлення структурованих оголошень про втрату та про знахідку.

На початковому етапі проведено аналіз предметної області, який показав, що наявні канали публікації оголошень розрізнені, оголошення не структуровані, а зіставлення втрати та знахідки виконується вручну. Аналіз аналогів — сервісу PawBoost, бази Pet FBI та тематичних спільнот — показав, що жодне з рішень не виконує автоматичного зіставлення, не ранжує ймовірні збіги і не приховує контактні дані до підтвердження. На цій основі сформовано функціональні вимоги у формі восьми варіантів використання, нефункціональні вимоги і вимоги до інтерфейсу.

Виконано планування проєкту: масштаб визначено переліком робіт з восьми етапів, складено календарний план і реєстр ризиків. Розмір проєкту оцінено методом Use Case Points — близько 51 UCP, що відповідає орієнтовній трудомісткості близько 1020 людино-годин.

Спроектовано систему: побудовано модель предметної області з трьох понять — користувач, оголошення, зв'язок; архітектуру за принципом портів та адаптерів; структуру бази даних із трьох таблиць; виконано проєктування на рівні класів із діаграмою класів і розв'язанням циклічної залежності модулів через порт.

Ключовим інженерним результатом є алгоритм зіставлення оголошень. Обґрунтовано набір формалізованих ознак — вид тварини, географічна відстань (формула гаверсинуса) і різниця дат події — та лексикографічне правило ранжування, що відображає природний пріоритет ознак. Перелік кандидатів обчислюється на запит засобами бази даних, а остаточне рішення про збіг ухвалює людина, що знижує ризик хибних збігів. На основі проєктних рішень реалізовано вебзастосунок із використанням платформи Node.js, фреймворку NestJS, бібліотеки React, ORM Drizzle та системи керування базами даних PostgreSQL; реалізовано при-

ховування контактних даних у публічних поданнях і взаємне розкриття їх лише після підтвердження зв'язку.

Функціональне та нефункціональне тестування підтвердило коректність системи: автоматизований набір із 130 серверних і 44 клієнтських тестів проходить успішно. Досягнення мети підтверджено експериментально на модельному наборі даних: істинний збіг потрапляє у перші три позиції ранжованого переліку в усіх запитах ($\text{Hit}@3 = 1,00$), на першу позицію — у 82 % запитів, середній обернений ранг становить 0,91; ранжування зменшує обсяг перегляду оголошень приблизно у 100 разів порівняно з хронологічним переглядом, а час формування переліку для 50 000 оголошень не перевищує 50 мс, що відповідає вимозі продуктивності у 2 секунди.

Практична цінність розробленої системи полягає в автоматизації найбільш трудомісткого етапу пошуку — виявлення ймовірного збігу — та у захисті контактних даних до підтвердження зв'язку. Поставлені в роботі задачі виконано; розроблений вебзастосунок може бути використаний волонтерськими спільнотами та притулками для автоматизації первинного зіставлення оголошень про загублених і знайдених тварин.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Weiss E., Slater M., Lord L. Frequency of Lost Dogs and Cats in the United States and the Methods Used to Locate Them. *Animals*. 2012. Vol. 2, No. 2. P. 301–315. <https://doi.org/10.3390/ani2020301> (дата звернення: 03.02.2026).
2. Lord L. K., Wittum T. E., Ferketich A. K., Funk J. A., Rajala-Schultz P. J. Search methods that people use to find owners of lost pets. *Journal of the American Veterinary Medical Association*. 2007. Vol. 230, No. 12. P. 1835–1840. <https://doi.org/10.2460/javma.230.12.1835> (дата звернення: 03.02.2026).
3. PawBoost: Lost & Found Pets. URL: <https://www.pawboost.com> (дата звернення: 05.02.2026).
4. Pet FBI: Lost and Found Pets Database. URL: <https://petfbi.org> (дата звернення: 05.02.2026).
5. OLX. Розділ оголошень про загублених і знайдених тварин. URL: <https://www.olx.ua> (дата звернення: 05.02.2026).
6. PostgreSQL Global Development Group. PostgreSQL Documentation. URL: <https://www.postgresql.org/docs/> (дата звернення: 28.02.2026).
7. Sinnott R. W. Virtues of the Haversine. *Sky and Telescope*. 1984. Vol. 68, No. 2. P. 159.
8. Fishburn P. C. Lexicographic Orders, Utilities and Decision Rules: A Survey. *Management Science*. 1974. Vol. 20, No. 11. P. 1442–1471. <https://doi.org/10.1287/mnsc.20.11.1442> (дата звернення: 19.02.2026).
9. OpenJS Foundation. Node.js Documentation. URL: <https://nodejs.org/docs/latest/api/> (дата звернення: 09.03.2026).
10. NestJS. Documentation. URL: <https://docs.nestjs.com> (дата звернення: 09.03.2026).
11. Drizzle Team. Drizzle ORM Documentation. URL: <https://orm.drizzle.team/docs/overview> (дата звернення: 09.03.2026).
12. Colin McDonnell. Zod Documentation. URL: <https://zod.dev> (дата звернення: 10.03.2026).

13. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT) : RFC 7519. Internet Engineering Task Force, 2015. URL: <https://www.rfc-editor.org/rfc/rfc7519> (дата звернення: 10.03.2026).

14. Meta Open Source. React Documentation. URL: <https://react.dev> (дата звернення: 23.03.2026).

ДОДАТОК А

Лістинги програмного коду

Лістинг А.1 – Програмна реалізація схеми бази даних

```
export const users = pgTable('users', {
  id: uuid('id').primaryKey().defaultRandom(),
  email: text('email').notNull().unique(),
  passwordHash: text('password_hash').notNull(),
  createdAt: timestamp('created_at', { withTimezone: true })
    .notNull().defaultNow(),
});

export const reports = pgTable('reports', {
  id: uuid('id').primaryKey().defaultRandom(),
  kind: text('kind').notNull(),
  reporterId: uuid('reporter_id').notNull().references(() => users.id),
  status: text('status').notNull(),
  species: text('species').notNull(),
  breed: text('breed'),
  name: text('name'),
  color: text('color'),
  description: text('description'),
  photoKey: text('photo_key'),
  contactPhone: text('contact_phone'),
  contactEmail: text('contact_email'),
  lat: doublePrecision('lat').notNull(),
  lng: doublePrecision('lng').notNull(),
  eventDate: timestamp('event_date', { withTimezone: true }).notNull(),
  createdAt: timestamp('created_at', { withTimezone: true })
    .notNull().defaultNow(),
  updatedAt: timestamp('updated_at', { withTimezone: true })
    .notNull().defaultNow(),
});

export const matches = pgTable('matches', {
  id: uuid('id').primaryKey().defaultRandom(),
  lostReportId: uuid('lost_report_id').notNull().references(() => reports.id),
  foundReportId: uuid('found_report_id').notNull().references(() => reports.id),
  proposedBy: uuid('proposed_by').notNull().references(() => users.id),
  status: text('status').notNull(),
  createdAt: timestamp('created_at', { withTimezone: true })
    .notNull().defaultNow(),
  resolvedAt: timestamp('resolved_at', { withTimezone: true }),
}, (t) => ({
  lostFoundUnique: unique('matches_lost_found_unique')
    .on(t.lostReportId, t.foundReportId),
}));
```

Лістинг А.2 – Програмна реалізація сервісу зіставлення оголошень

```
findCandidates(subject: ReportRecord): ResultAsync<CandidateRecord[], DbError> {
  const oppositeKind = subject.kind === 'lost' ? 'found' : 'lost';
  const distanceKm = this.haversineKm(subject.lat, subject.lng);
  const speciesMatch = sql<boolean>`${reports.species} = ${subject.species}`;
  const subjectDateIso = subject.eventDate.toISOString();
  const daysApart = sql<number>`abs(extract(epoch from
    (${reports.eventDate} - ${subjectDateIso}::timestampz)) / 86400)`;

  const query = this.db
    .select({ row: reports, distanceKm, speciesMatch, daysApart })
    .from(reports)
    .where(and(eq(reports.kind, oppositeKind), ne(reports.id, subject.id)))
    .orderBy(desc(speciesMatch), distanceKm, daysApart);

  return ResultAsync.fromPromise(query, (cause) => dbError(cause)).map(
```

```

(rows) =>
  rows.map((r) => ({
    report: toRecord(r.row),
    distanceKm: Number(r.distanceKm),
    speciesMatch: r.speciesMatch,
  })),
);
}

```

Лістинг А.3 – Програмна реалізація обчислення відстані

```

const EARTH_RADIUS_KM = 6371;

private haversineKm(lat: number, lng: number): SQL<number> {
  return sql<number>`${EARTH_RADIUS_KM} * acos(
    least(1, greatest(-1,
      cos(radians(${lat})) * cos(radians(${reports.lat}))
      * cos(radians(${reports.lng}) - radians(${lng}))
      + sin(radians(${lat})) * sin(radians(${reports.lat}))
    ))
  )`;
}

```

Лістинг А.4 – Програмна реалізація сервісу підтвердження зв'язку

```

confirm(actorId: string, matchId: string) {
  return this.decide(actorId, matchId, 'confirmed')
    .andThen((record) => this.reveal(record));
}

private decide(actorId: string, matchId: string,
  status: 'confirmed' | 'rejected') {
  return this.repo.findById(matchId).andThen((match) => {
    if (match === null) return errAsync(notFound('match', matchId));
    const isStillPending = match.status === 'proposed';
    if (!isStillPending)
      return errAsync(
        conflict('only a proposed match can be confirmed or rejected'));
    return this.authorizeDecider(actorId, match)
      .andThen(() => this.repo.resolve(match.id, status));
  });
}

private authorizeDecider(actorId: string, match: MatchRecord) {
  return ResultAsync.combine([
    this.reports.getRecord(match.lostReportId),
    this.reports.getRecord(match.foundReportId),
  ]).andThen(([lost, found]) => {
    const proposerOwnsLostSide = lost.reporterId === match.proposedBy;
    const decidingReporterId = proposerOwnsLostSide
      ? found.reporterId : lost.reporterId;
    const isNonProposingReporter = actorId === decidingReporterId;
    if (!isNonProposingReporter)
      return errAsync(
        forbidden('only the non-proposing reporter may decide this match'));
    return okAsync<true>(true);
  });
}

private reveal(record: MatchRecord) {
  return ResultAsync.combine([
    this.reports.revealContact(record.lostReportId),
    this.reports.revealContact(record.foundReportId),
  ]).map(([lostReport, foundReport]) => ({
    ...toView(record), lostReport, foundReport,
  }));
}

```
